

Breno Cerqueira Reis Nakamura

Implementação do Compilador C-

São José dos Campos - Brasil

Junho de 2026

Breno Cerqueira Reis Nakamura

Implementação do Compilador C-

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Junho de 2026

Lista de ilustrações

Figura 1 – Diagrama de blocos do processador.	10
Figura 2 – Diagrama de blocos do Módulo de Entrada e Saída.	11
Figura 3 – Diagrama de blocos da Unidade de Processamento.	12
Figura 4 – Diagrama de blocos da Unidade de Controle.	13
Figura 5 – Datapath geral do projeto	14
Figura 6 – Formatos das instruções tipo I, J e R	24
Figura 7 – Diagrama de blocos do módulo de análise.	30
Figura 8 – Diagrama de atividades do módulo de análise.	31
Figura 9 – Diagrama de blocos do Gerador de Código Intermediário.	42
Figura 10 – Diagrama de atividades do Gerador de Código Intermediário.	43
Figura 11 – Diagrama de blocos do Gerador de Código Assembly.	44
Figura 12 – Diagrama de atividades do Gerador de Código Assembly.	46
Figura 13 – Diagrama de blocos do Gerador de Código Binário.	48
Figura 14 – Diagrama de atividades do Gerador de Código Binário.	49

Lista de tabelas

Tabela 1 – Sinais de Controle de cada instrução	17
Tabela 2 – Descrição do ALUOp	19
Tabela 4 – Instruções do tipo R	24
Tabela 3 – Conjunto de instruções do processador	25
Tabela 5 – Instruções do tipo I	26
Tabela 6 – Instruções do tipo J	26
Tabela 7 – Tokens reconhecidos pelo analisador léxico.	33
Tabela 8 – Tipos de quádruplas definidos no projeto.	51

Sumário

1	INTRODUÇÃO	7
2	O PROCESSADOR	9
2.1	Diagrama de Blocos do Processador	9
2.2	Componentes do Processador	14
2.2.1	Caminho de Dados	14
2.2.2	Divisor de Clock	14
2.2.3	Debouncer	15
2.2.4	Contador de Programa (PC)	16
2.2.5	Memória de Instrução	16
2.2.6	Unidade de Controle	17
2.2.7	Unidade de Controle da ULA	19
2.2.8	Extensor de sinal	20
2.2.9	Banco de Registradores	20
2.2.10	Multiplexadores	21
2.2.11	Unidade Lógica e Aritmética (ULA)	21
2.2.12	Memória de Dados	22
2.2.13	Processador	23
2.3	Conjunto de Instruções	23
2.4	Organização da Memória	26
2.4.1	Memória de Instruções	26
2.4.2	Memória de Dados	27
3	COMPILADOR: FASE DE ANÁLISE	29
3.1	Modelagem	29
3.1.1	Diagrama de Blocos	29
3.1.2	Diagrama de Atividades	31
3.2	Análise Léxica	32
3.3	Análise Sintática	34
3.4	Análise Semântica	36
3.4.1	Tabela de Símbolos	36
3.4.2	Validações Realizadas	37
3.4.3	Funções Nativas e Validação da Função main	38
4	COMPILADOR: FASE DE SÍNTESE	41
4.1	Modelagem	41

4.1.1	Modelagem do Gerador de Código Intermediário	41
4.1.1.1	Diagrama de Blocos	41
4.1.2	Diagrama de Atividades	42
4.1.3	Modelagem do Gerador de Código Assembly	44
4.1.3.1	Diagrama de Blocos	44
4.1.3.2	Diagrama de Atividades	45
4.1.4	Modelagem do Gerador de Código Binário	47
4.1.4.1	Diagrama de Blocos	47
4.1.4.2	Diagrama de Atividades	49
4.2	Geração do Código Intermediário	50
4.3	Geração do Código Assembly	53
4.4	Geração do Código Executável	55
4.4.1	Primeira Passagem: Resolução de Rótulos	56
4.4.2	Segunda Passagem: Codificação Binária	56
4.4.3	Saída Gerada	57
4.5	Gerenciamento de Memória	58
5	EXEMPLOS	61
5.1	Exemplo 1 – Fatorial	61
5.2	Exemplo 2 – GCD	68
5.3	Exemplo 3 – Sort	75
6	CONCLUSÃO	89
6.1	Dificuldades Encontradas	89
6.2	Destaques	89
6.3	Trabalhos Futuros	90
	REFERÊNCIAS	91

1 Introdução

A construção de software moderno seria inviável sem a existência de ferramentas capazes de traduzir a intenção do programador em instruções executáveis pelo hardware. Escrever programas diretamente em linguagem de máquina, embora tecnicamente possível, é uma tarefa árdua, sujeita a erros e fortemente acoplada às particularidades de cada arquitetura. Para abstrair essa complexidade, foram criadas as linguagens de programação de alto nível, que aproximam a escrita de código da forma como os seres humanos descrevem problemas e algoritmos. Tornou-se necessário, então, um mecanismo que fizesse a ponte entre esses dois mundos: os compiladores.

De acordo com Aho et al. (1), um compilador é um programa que lê um código escrito em uma linguagem, chamada linguagem fonte, e o traduz para um código equivalente em outra linguagem, chamada linguagem alvo, preservando a semântica do programa original. Em geral, essa tradução parte de uma linguagem de alto nível em direção à linguagem de máquina de um processador específico, permitindo que o mesmo programa, com pequenos ajustes, possa ser executado em arquiteturas distintas. Os compiladores ocupam, portanto, um papel central na computação contemporânea: são eles que viabilizam a portabilidade, a produtividade do desenvolvimento e o desacoplamento entre o projeto de software e os detalhes do hardware subjacente.

Neste contexto, o presente projeto tem como objetivo o desenvolvimento de um compilador completo para a linguagem C-, um subconjunto reduzido da linguagem C, tendo como arquitetura alvo o processador implementado na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores. Trata-se de um processador baseado na arquitetura MIPS monociclo, com conjunto de instruções RISC e organização de memória do tipo Harvard, ou seja, com memórias de instruções e de dados fisicamente separadas. O compilador foi construído contemplando suas duas grandes etapas: a fase de análise, responsável pela verificação léxica, sintática e semântica do código fonte, e a fase de síntese, responsável pela geração de código intermediário, código assembly e código executável para a arquitetura alvo.

O restante deste relatório está organizado da seguinte forma. O Capítulo 2 descreve o processador utilizado como alvo da compilação, apresentando seu diagrama de blocos, seus componentes, seu conjunto de instruções e a organização de memória adotada. O Capítulo 3 aborda a fase de análise do compilador, detalhando a modelagem em SysML e a implementação das análises léxica, sintática e semântica. O Capítulo 4 apresenta a fase de síntese, incluindo a geração do código intermediário em quádruplas, a tradução para o assembly do processador, a montagem do código executável e o gerenciamento de

memória. No Capítulo 5 são apresentados exemplos de programas em C- compilados pela ferramenta, evidenciando a correspondência entre o código fonte, o código intermediário e o código assembly gerado. Por fim, o Capítulo 6 traz as considerações finais, discutindo as principais dificuldades enfrentadas, os destaques do trabalho e possíveis desdobramentos futuros.

2 O PROCESSADOR

O processador desenvolvido tem, como referência, a arquitetura MIPS, com instruções RISC, executadas de maneira individual e em monociclo. Com essa configuração, prioriza-se a simplicidade e eficiência do processador, utilizando um conjunto reduzido de instruções, com tamanho fixo e formato padronizado, facilitando a implementação. Em relação à organização da memória, foi adotado o modelo Harvard, no qual as memórias de dados e de instruções são separadas fisicamente, permitindo acessos paralelos a dados e instruções, melhorando o desempenho e evitando gargalos.

Todas as instruções possuem tamanho fixo de 32 bits, assim como no MIPS desenvolvido por Patterson (2), o que garante previsibilidade na decodificação das instruções. Além disso, o conjunto de registradores é composto por 32 registradores de uso geral, oferecendo a flexibilidade necessária no armazenamento temporário de dados durante a execução das instruções.

Os modos de endereçamento são os mesmos da arquitetura MIPS padrão. Portanto, a arquitetura contará com os seguintes modos de endereçamento: endereçamento imediato, endereçamento em registrador, endereçamento de base, endereçamento relativo ao PC e endereçamento pseudodireto.

O contador de programa (PC) é incrementado em uma unidade após a execução de cada instrução, ou seja, $PC = PC + 1$. Esse comportamento ocorre porque a memória utilizada é do tipo *word-addressable*, na qual cada posição armazena uma palavra de 32 bits — o mesmo tamanho fixo das instruções —, permitindo que o PC aponte diretamente para o próximo endereço de instrução sem a necessidade de incrementos adicionais.

As entradas (*input*) do processador serão representadas pelos valores fornecidos por meio dos botões e *switches* disponíveis no *kit* FPGA, enquanto as saídas (*output*) serão exibidas por meio dos dígitos no *display* de 7 segmentos.

2.1 Diagrama de Blocos do Processador

Nesta seção são descritos os diagramas de blocos referentes ao processador desenvolvido. Inicialmente, a Figura 1 apresenta a organização geral do processador, evidenciando seus principais módulos: Unidade de Controle, Unidade de Processamento, Módulo de Entrada e Saída e Memória de Instruções. Em seguida, a Figura 2 detalha a estrutura interna do módulo responsável pela interface de comunicação com o meio externo. Posteriormente, a Figura 3 ilustra os componentes que constituem a Unidade de Processamento. Finalmente, a Figura 4 apresenta a organização em blocos da Unidade de Controle.

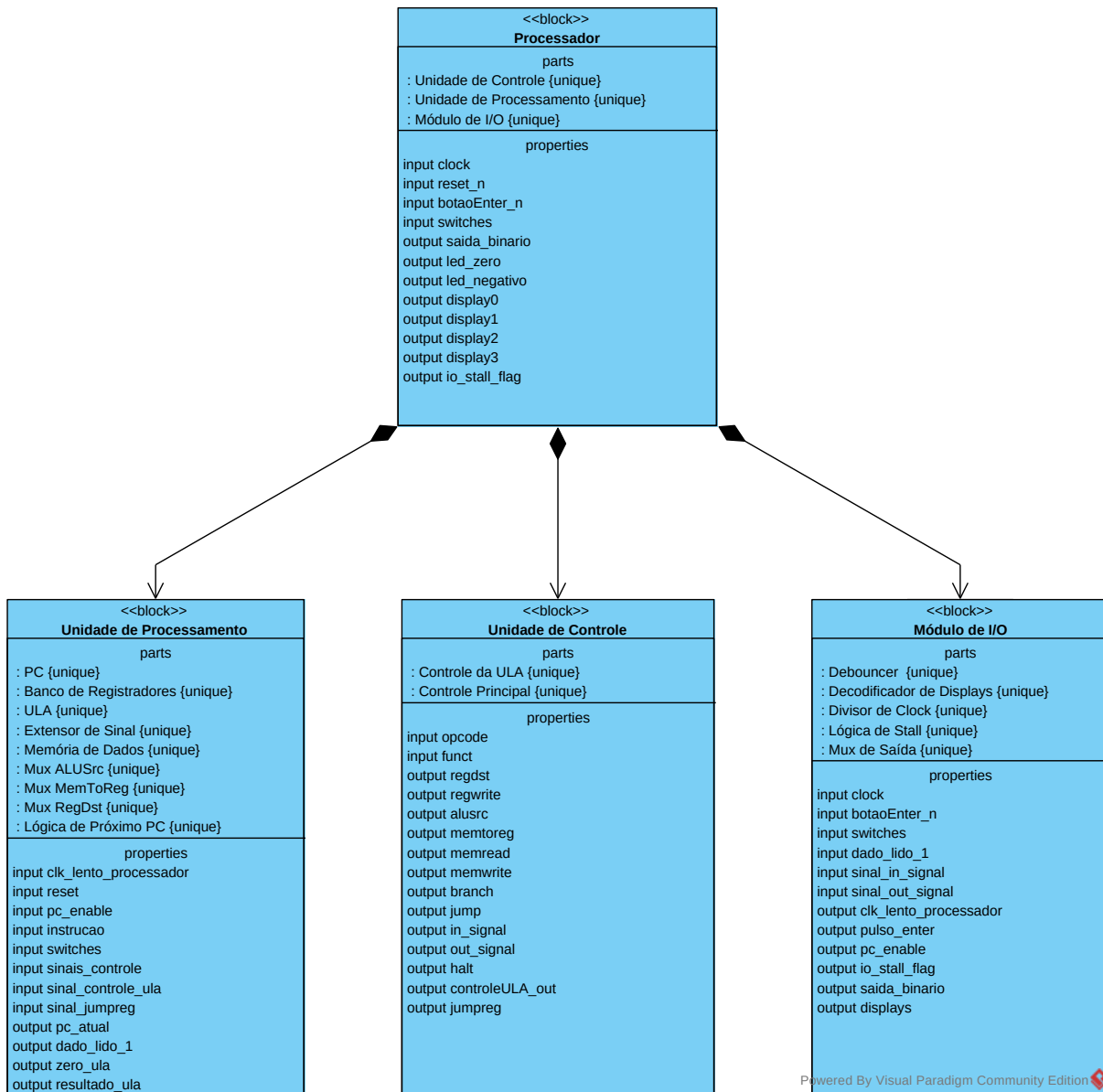


Figura 1 – Diagrama de blocos do processador.

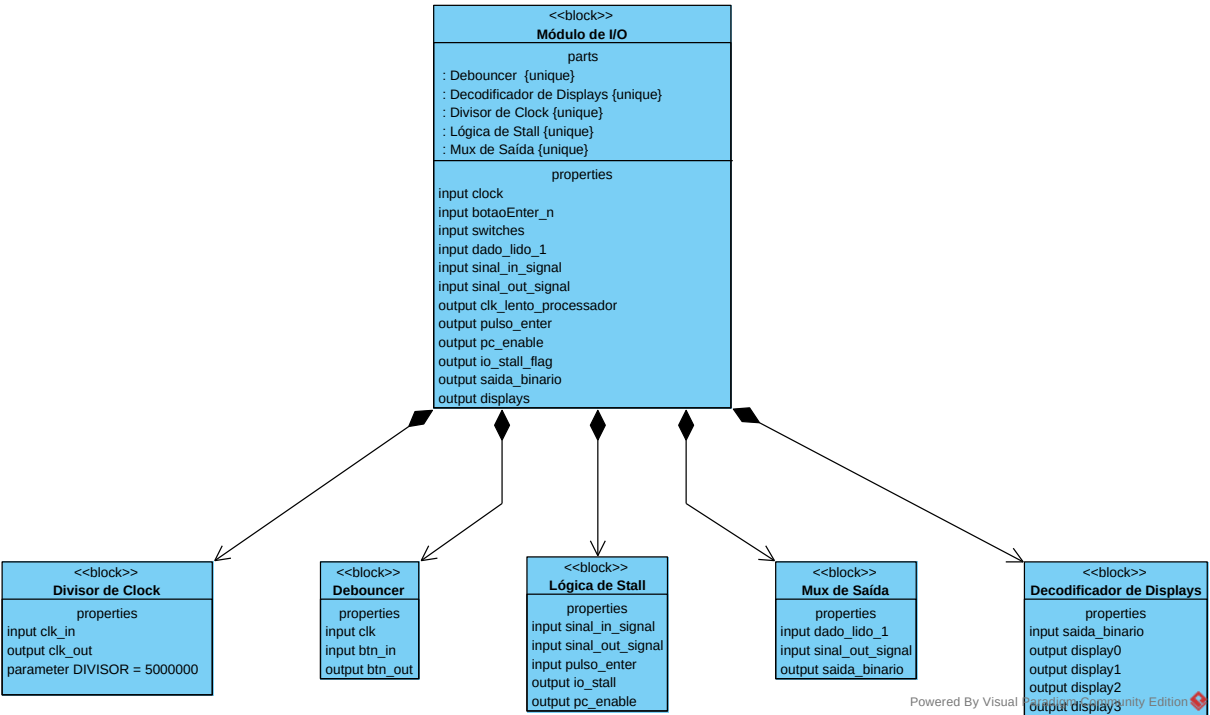


Figura 2 – Diagrama de blocos do Módulo de Entrada e Saída.

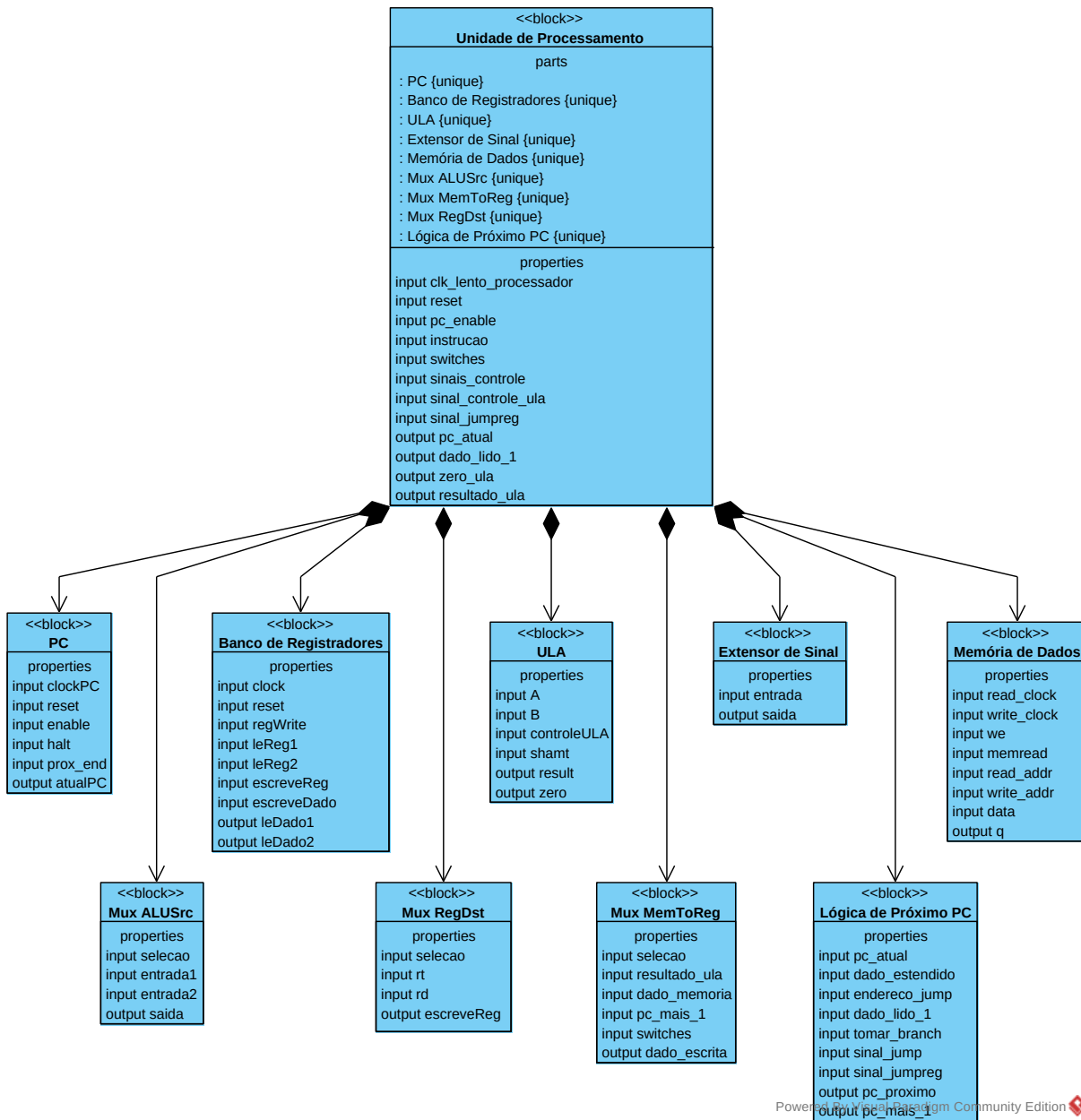


Figura 3 – Diagrama de blocos da Unidade de Processamento.

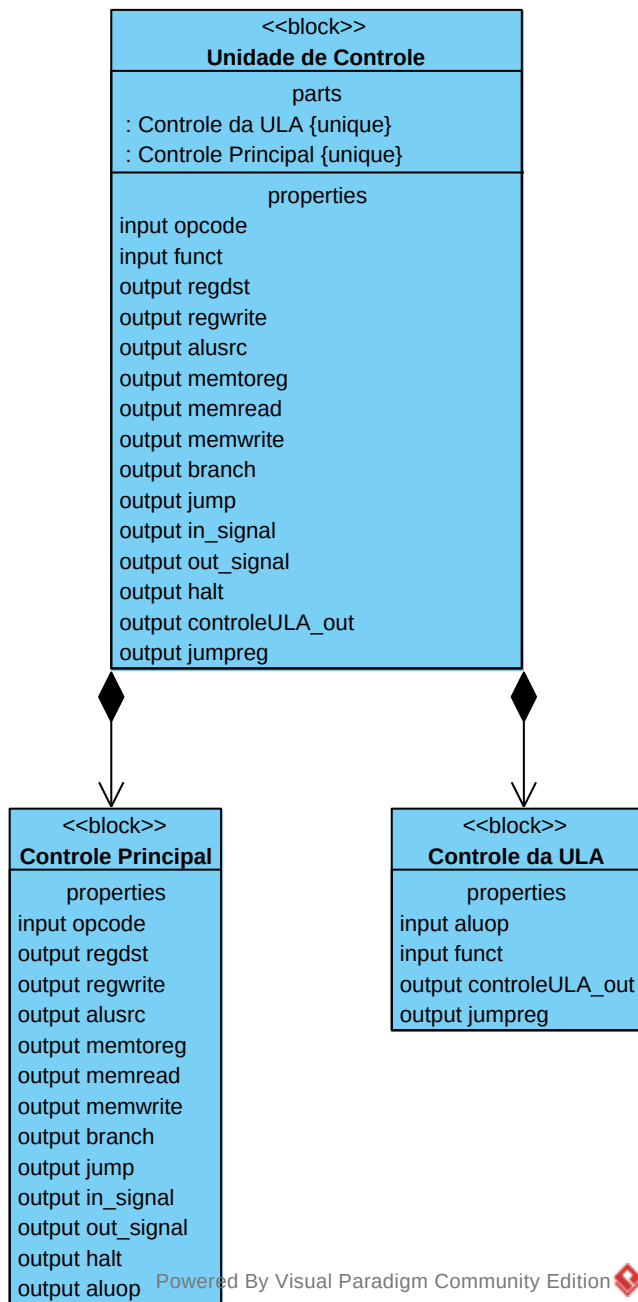


Figura 4 – Diagrama de blocos da Unidade de Controle.

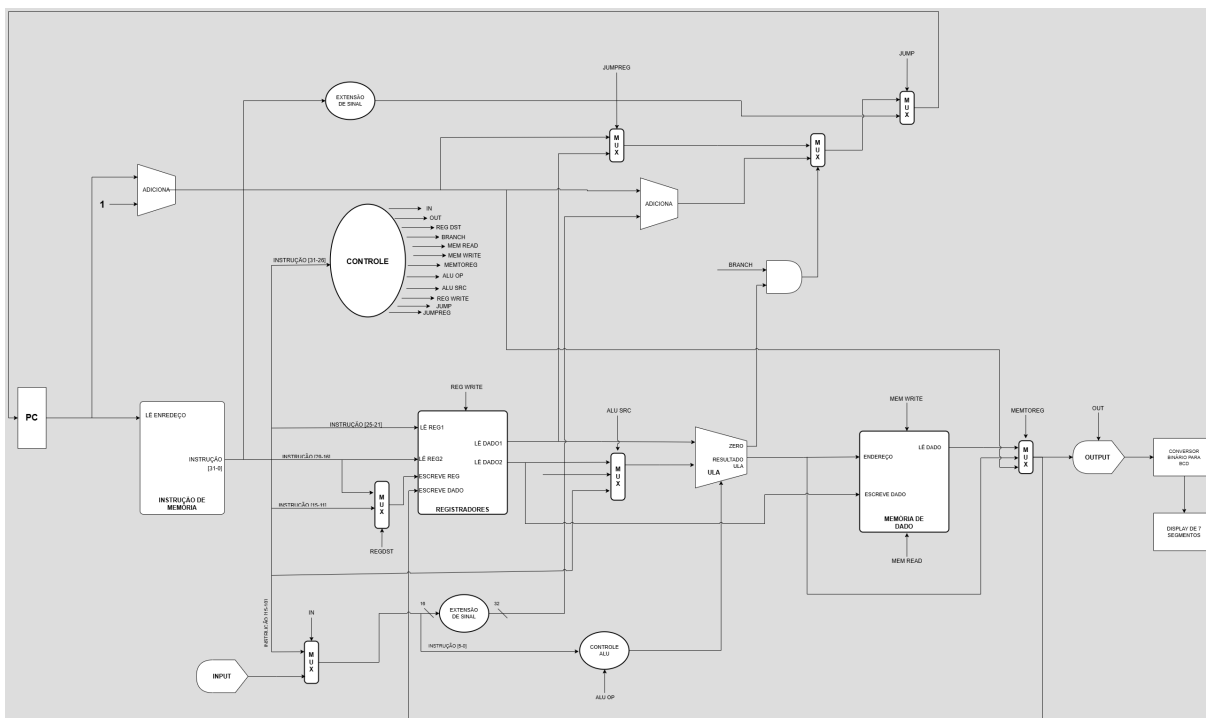
2.2 Componentes do Processador

2.2.1 Caminho de Dados

O *datapath* (caminho de dados) representa o caminho físico que os dados percorrem dentro do processador durante a execução de determinada instrução. Dentre os componentes do datapath, destacam-se: a ULA (Unidade Lógica Aritmética), para realizar operações lógicas e aritméticas; banco de registradores; unidade de controle, que envia sinais para os multiplexadores a partir do *opcode* da instrução a ser executada. Vale destacar a divisão de memória em memória de instrução e memória de dados, seguindo a arquitetura de Harvard.

A Figura 5 ilustra o *datapath* correspondente ao presente projeto, considerando as escolhas feitas para o processador, possuindo, assim, algumas mudanças em relação ao *datapath* da arquitetura MIPS padrão, como por exemplo, o incremento do PC (*Program Counter*) em $PC + 1$.

Figura 5 – Datapath geral do projeto



Fonte: O autor

2.2.2 Divisor de Clock

O módulo `divisorClock` tem como finalidade gerar um sinal de clock com frequência reduzida a partir do clock principal fornecido pelo FPGA. Essa redução é útil para permitir a visualização mais clara do funcionamento do processador durante testes físicos,

especialmente com LEDs, displays e botões.

A implementação realiza a contagem de ciclos do clock de entrada (`clk_in`) até atingir o valor definido pelo parâmetro `DIVISOR`. A cada ciclo, um contador é incrementado, e o sinal de saída (`clk_out`) é definido como alto (1) na primeira metade da contagem e baixo (0) na segunda metade. Isso gera um sinal de onda quadrada com *ciclo ativo de 50%* e *frequência igual a $\text{clk_in} / \text{DIVISOR}$* .

O código foi parametrizado utilizando a diretiva `$clog2(DIVISOR)` para que o tamanho do registrador `counter` seja automaticamente ajustado à largura mínima necessária para representar o valor do divisor, garantindo maior flexibilidade e economia de recursos lógicos.

O módulo foi desenvolvido com base em um exemplo adaptado do site [FPGA4Students \(3\)](#), que apresenta um projeto semelhante de divisor de clock para uso geral em FPGAs.

Com `DIVISOR = 5_000_000` e `clk_in = 50 MHz`, por exemplo, o sinal `clk_out` resultante terá frequência de aproximadamente 10 Hz, permitindo a visualização clara da alternância de estados em testes físicos com o processador.

2.2.3 Debouncer

O módulo *Debouncer* é um componente utilitário essencial, projetado para filtrar o ruído elétrico inerente aos botões físicos e converter o sinal instável de entrada em um pulso digital limpo, com a duração de exatamente um ciclo de clock. Essa funcionalidade é crucial para garantir que o processador interprete um único apertado de botão como um único evento, evitando múltiplas ativações indesejadas que ocorreriam devido às oscilações mecânicas do botão.

Para otimizar o projeto tanto para a validação em software quanto para a implementação em hardware, o módulo utiliza um bloco `generate` controlado pelo parâmetro `SIMULATION`. Essa estrutura permite que duas versões distintas do circuito sejam sintetizadas. Para a simulação, com `SIMULATION = 1`, onde os sinais de entrada são ideais e não possuem ruído, é gerada uma versão simplificada que apenas detecta a borda de subida do sinal. Para a implementação no FPGA, é gerada a versão completa com a lógica de filtragem de ruído.

No modo de implementação em FPGA (`SIMULATION = 0`), a lógica de *debounce* é baseada em um contador síncrono. Quando o estado do botão de entrada (`btn_in`) muda, o contador é iniciado. Ele mede o tempo em que o sinal de entrada permanece estável. Somente após o sinal se manter constante por um período pré-definido de 10 milissegundos (`DEBOUNCE_TIME`), o novo valor é considerado válido e armazenado internamente. Na sequência, um segundo estágio de *flip-flops* atua como um detector de borda sobre este sinal já estabilizado, produzindo na saída (`btn_out`) um pulso único e limpo, perfeitamente

sincronizado com o clock do sistema.

2.2.4 Contador de Programa (PC)

O módulo PC (*Program Counter*) é responsável por armazenar o endereço da próxima instrução a ser executada pelo processador. Sua principal função é controlar o fluxo sequencial do programa, além de possibilitar desvios e saltos quando necessário.

O funcionamento do módulo é controlado por quatro sinais de entrada principais: `clockPC`, `reset`, `enable` e `halt`. O sinal `clockPC` é o clock utilizado para atualização síncrona do contador. O sinal `reset`, por sua vez, atua de forma assíncrona e quando ativado, força imediatamente o valor do contador a zero, reiniciando a execução do programa. A entrada `enable` permite habilitar ou desabilitar a atualização do contador. Quando `enable` está em nível baixo, o valor armazenado no PC permanece inalterado, mesmo com a mudança do clock. Já o sinal `halt` serve para interromper a execução do programa. Quando `halt` está em nível alto, o PC também mantém seu valor atual, mesmo que o sinal `enable` esteja ativado.

O endereço da próxima instrução é fornecido pela entrada `prox_end`, que pode representar tanto o endereço sequencial ($PC + 1$) quanto um endereço oriundo de uma instrução de salto ou desvio. A saída `atualPC` reflete o valor atual armazenado no contador, sendo usada como endereço de leitura da próxima instrução na memória ROM.

Dessa forma, o PC permite tanto o avanço sequencial das instruções quanto a alteração do fluxo de execução, quando necessário.

2.2.5 Memória de Instrução

O módulo `memoriaROM` implementa a memória apenas de leitura (ROM) do processador, responsável por armazenar o programa que será executado. Sua estrutura foi baseada em um *template* de memória do próprio software Quartus Prime, o que garante uma inferência eficiente dos blocos de memória dedicados (*Block RAM* - BRAM) disponíveis no FPGA, otimizando o uso de recursos. A memória foi projetada para armazenar 256 palavras de 32 bits, endereçadas pelos 8 bits menos significativos do PC.

Uma característica fundamental deste módulo é a sua capacidade de inicializar seu conteúdo a partir de um arquivo externo. Através da diretiva de síntese `$readmemb("codigo.mif", rom)`, o programa, previamente compilado para código de máquina, é lido do arquivo `codigo.mif` e “gravado” na estrutura da memória durante o processo de compilação do projeto. Isso significa que o programa já está presente no hardware assim que o FPGA é ligado.

Para otimizar o projeto tanto para a simulação quanto para a implementação física, foi utilizado um bloco `generate` controlado pelo parâmetro `SIMULATION`. Esta

técnica permite que duas versões distintas do circuito sejam geradas a partir do mesmo código-fonte. Quando **SIMULATION** é definido como 1, é sintetizada uma versão com leitura combinacional (`assign q = rom[addr];`). Neste modo, a instrução é disponibilizada na saída instantaneamente assim que o endereço do PC muda, o que acelera e simplifica a depuração no ambiente ModelSim.

Por outro lado, quando **SIMULATION** = 0 (para a implementação no FPGA), o módulo gera uma versão com leitura síncrona sinal de clock `clk`, comportamento mais próximo do que ocorre em FPGAs reais. A instrução é carregada na borda de subida do clock.

2.2.6 Unidade de Controle

O módulo **controle** constitui a Unidade de Controle principal do processador. Sua função é decodificar o campo **opcode** (6 bits) da instrução em execução e gerar os sinais de controle responsáveis por coordenar as operações do datapath durante um ciclo de clock.

Seu funcionamento é puramente combinacional, implementado com um bloco **always_comb**, utilizando uma estrutura **case** que atribui valores específicos aos sinais de controle com base no **opcode** recebido. Esses sinais, apresentados em a seguir, são distribuídos para os diversos módulos do processador conforme a necessidade de cada instrução.

A [Tabela 1](#) exibe o valor atribuído a cada sinal de controle para cada instrução implementada, permitindo visualizar o comportamento da arquitetura de forma clara e sistemática.

Tabela 1 – Sinais de Controle de cada instrução

INSTRUÇÃO	OPCODE	RegDst	RegWrite	ALUSrc	MemToReg	MemRead	MemWrite	Branch	Jump	IN	OUT	HALT	ALUOp
Tipo R	000000	01	1	0	00	0	0	0	0	0	0	0	tipoR
JR	000000	11	0	X	XX	0	0	0	1	0	0	0	X
ADDI	000001	00	1	1	00	0	0	0	0	0	0	0	SOMA
SUBI	000010	00	1	1	00	0	0	0	0	0	0	0	SUB
ANDI	000011	00	1	1	00	0	0	0	0	0	0	0	AND
ORI	000100	00	1	1	00	0	0	0	0	0	0	0	OR
LW	000101	00	1	1	01	1	0	0	0	0	0	0	SOMA
SW	000110	XX	0	1	XX	0	1	0	0	0	0	0	SOMA
BEQ	000111	XX	0	0	XX	0	0	1	0	0	0	0	SUB
BNE	001000	XX	0	0	XX	0	0	1	0	0	0	0	SUB
BLT	001001	XX	0	0	XX	0	0	1	0	0	0	0	SLT
BGT	001010	XX	0	0	XX	0	0	1	0	0	0	0	SGT
BLE	001011	XX	0	0	XX	0	0	1	0	0	0	0	SLE
BGE	001100	XX	0	0	XX	0	0	1	0	0	0	0	SGE
IN	001101	00	1	X	11	0	0	0	0	1	0	0	X
OUT	001110	XX	0	X	XX	0	0	0	0	0	1	0	X
J	010000	XX	0	X	XX	0	0	0	1	0	0	0	X
JAL	010001	10	1	X	10	0	0	0	1	0	0	0	X
NOP	010010	XX	0	X	XX	0	0	0	0	0	0	0	X
HALT	010011	XX	0	X	XX	0	0	0	0	0	0	1	X

Fonte: O Autor

A seguir, são descritos os sinais de controle utilizados no processador desenvolvido, destacando sua função e codificação, quando aplicável:

- **RegDst** (2 bits): Define qual registrador será utilizado como destino na escrita do banco de registradores.
 - 00 – Registrador **rt** (instruções do tipo I)
 - 01 – Registrador **rd** (instruções do tipo R)
 - 10 – Registrador 31 (utilizado em instruções como **JAL**)
 - 11 – Nenhuma escrita (sem registrador destino)
- **RegWrite**: Habilita a escrita no banco de registradores.
 - 0 – Escrita desabilitada
 - 1 – Escrita habilitada
- **ALUSrc**: Seleciona a origem do segundo operando da ULA.
 - 0 – Segundo operando vem do registrador **rt**
 - 1 – Segundo operando é um valor imediato (oriundo do extensor)
- **MemToReg** (2 bits): Define a origem do dado que será escrito no banco de registradores.
 - 00 – Resultado da ULA
 - 01 – Dado proveniente da memória (instrução **LW**)
 - 10 – Valor de **PC + 1** (utilizado em **JAL**)
 - 11 – Dado de entrada externa (instrução **IN**) ou valor constante
- **MemRead**: Habilita a leitura da memória de dados.
 - 0 – Desabilitada
 - 1 – Habilitada (o conteúdo da memória no endereço especificado será colocado na saída)
- **MemWrite**: Habilita a escrita na memória de dados.
 - 0 – Desabilitada
 - 1 – Habilitada (o valor na entrada de dados será escrito no endereço especificado)
- **Branch**: Indica que a instrução é de desvio condicional (como **BEQ**, **BNE**, **BLT**, etc.). Esse sinal ativa a lógica responsável por decidir se o **PC** será alterado com base no resultado da comparação.
- **Jump**: Sinaliza que a instrução é de salto incondicional (**J**, **JAL**).
 - 0 – **PC** avança sequencialmente

- 1 – Salto incondicional para endereço especificado na instrução
- **IN**: Habilita a operação de entrada de dados externos (instrução **IN**).
 - 0 – Nenhuma entrada
 - 1 – Dado de entrada será capturado e escrito no registrador
- **OUT**: Habilita a operação de saída de dados (instrução **OUT**).
 - 0 – Nenhuma saída
 - 1 – Valor do registrador será enviado à saída
- **HALT**: Interrompe a execução do processador.
 - 0 – Execução contínua
 - 1 – Interrupção do contador de programa (processador entra em estado de parada)
- **ALUOp** (4 bits): Define qual operação a ULA deve realizar. O valor deste sinal é interpretado posteriormente pela unidade **ULAcontrol**, que determina a operação exata com base no campo **funct**, quando necessário. A [Tabela 2](#) apresenta os valores possíveis do sinal **aluop**, acompanhados de sua descrição e operação correspondente na Unidade Lógica e Aritmética (ULA).

Tabela 2 – Descrição do ALUOp

OPERAÇÃO	CÓDIGO	DESCRIÇÃO
tipoR	0000	Dependerá do FUNCT
SOMA	0001	Operação de soma
SUB	0010	Operação de subtração
AND	0011	Operação AND
OR	0100	Operação OR
SLT	0101	Comparação "menor que"
SGT	0110	Comparação "maior que"
SLE	0111	Comparação "menor ou igual"
SGE	1000	Comparação "maior ou igual"

Fonte: O Autor

2.2.7 Unidade de Controle da ULA

O módulo **ULAcontrol** atua como um decodificador secundário e especializado, que opera em conjunto com a Unidade de Controle principal para gerar o sinal de controle final de 4 bits (**controleULA_out**) que comanda diretamente a ULA. Sua função é traduzir o comando de operação genérico, recebido através do sinal **aluop**, na instrução específica que a ULA deve executar. Para isso, o módulo recebe como entradas o sinal **aluop** de 4 bits,

vindo da Unidade de Controle principal, e o campo **funct** de 6 bits, extraído diretamente da instrução.

A implementação utiliza uma lógica de decodificação em dois níveis, implementada de forma puramente combinacional com um bloco **always_comb**. Primeiramente, a estrutura **case** avalia o sinal **aluop**. Se este indica uma instrução do tipo R (**ALUOP_TIPO_R**), a decisão sobre qual operação será realizada é tomada com base no valor de **funct**. Já para os demais casos, o próprio **aluop** determina diretamente o código da operação da ULA.

Além de gerar o comando para a ULA, este módulo tem a responsabilidade crucial de identificar a instrução JR (Jump Register). Ao decodificar o **funct** correspondente a esta instrução, ele ativa o sinal de saída de 1 bit **jumpreg**. Este sinal é essencial para a lógica de controle do datapath, pois comanda o multiplexador que seleciona o endereço de retorno armazenado em um registrador como o próximo valor do PC. Para facilitar a leitura do código, todas as operações possíveis da ULA foram definidas com nomes simbólicos utilizando **localparam**.

2.2.8 Extensor de sinal

O módulo **extensor** tem como função realizar a extensão de sinal de um valor de 16 bits para 32 bits, garantindo que o valor seja corretamente interpretado como positivo ou negativo, conforme a representação em complemento de dois. Esse processo é essencial para instruções que utilizam operandos imediatos na arquitetura MIPS, permitindo que operandos de 16 bits sejam utilizados em operações com dados de 32 bits no processador.

O funcionamento do módulo é combinacional. Ele verifica o bit mais significativo da entrada, que corresponde ao bit de sinal. Se esse bit for igual a 1, indicando um valor negativo, os 16 bits mais significativos da saída são preenchidos com '1', preservando o sinal negativo na conversão. Caso contrário, se o bit de sinal for zero, os bits mais altos da saída são preenchidos com '0', mantendo o valor positivo. Os 16 bits menos significativos da saída são simplesmente preenchidos com o valor da entrada.

Dessa forma, o módulo garante a correta propagação do sinal ao expandir valores de 16 para 32 bits.

2.2.9 Banco de Registradores

O banco de registradores é responsável pelo armazenamento temporário de dados e operandos durante a execução das instruções. No projeto implementado, o banco possui 32 registradores de 32 bits cada, seguindo a convenção da arquitetura MIPS, onde o registrador de índice zero é constante com valor zero.

O funcionamento do módulo é controlado por sinais de entrada. A escrita nos registradores ocorre na borda de subida do clock, desde que o sinal de controle **regWrite**

esteja ativo. O dado fornecido na entrada **escreveDado** é então armazenado no registrador indicado por **escreveReg**, exceto se for o registrador de número zero, que é protegido contra alterações. Além disso, o módulo possui um sinal de **reset** síncrono que, quando ativado, zera todo o conteúdo dos registradores.

As leituras dos dados dos registradores são realizadas de forma combinacional, ou seja, os valores dos registradores são imediatamente disponibilizados nas saídas **leDado1** e **leDado2** de acordo com os endereços especificados pelas entradas **leReg1** e **leReg2**, respectivamente. Dessa forma, o módulo permite duas leituras simultâneas e uma escrita por ciclo de clock.

2.2.10 Multiplexadores

Os multiplexadores são componentes fundamentais no datapath, responsáveis pela seleção de dados provenientes de diferentes fontes, permitindo que o sinal correto seja encaminhado para a próxima etapa do processamento, conforme o controle da Unidade de Controle.

Foram implementados dois tipos de multiplexadores: um multiplexador de 2 entradas (**mux2**) e um multiplexador de 3 entradas (**mux3**). Ambos operam de forma combinacional, ou seja, a saída é atualizada instantaneamente sempre que qualquer uma das entradas ou o sinal de seleção for alterado.

O módulo **mux2** possui duas entradas de 32 bits e um sinal de seleção de 1 bit. Quando o sinal de seleção é igual a '0', a saída recebe o valor da entrada **entrada1**. Quando o sinal é '1', a saída recebe o valor da entrada **entrada2**. Caso ocorra uma condição inválida (fora dos valores esperados), a saída é definida como zero.

O módulo **mux3** funciona de forma semelhante, mas com três entradas de 32 bits e um sinal de seleção de 2 bits. Dependendo do valor do sinal de seleção, a saída recebe o valor de uma das três entradas: **entrada1**, **entrada2** ou **entrada3**. Assim como no **mux2**, se o valor da seleção estiver fora do esperado, a saída é zerada.

Esses multiplexadores são essenciais para controlar o fluxo de dados dentro do datapath, permitindo que diferentes fontes alimentem os componentes da Unidade de Processamento de acordo com a operação a ser executada.

2.2.11 Unidade Lógica e Aritmética (ULA)

A Unidade Lógica e Aritmética (ULA) é responsável por executar as operações matemáticas, lógicas e de comparação do processador, de acordo com os sinais de controle recebidos. Ela recebe dois operandos de 32 bits, denominados **A** e **B**, além de um código de controle de 4 bits (**controleULA**), que define qual operação será realizada, e um sinal **shamt**, utilizado especificamente nas operações de deslocamento de bits.

O módulo realiza operações aritméticas como soma (**ADD**), subtração (**SUB**), multiplicação (**MUL**) e divisão (**DIV**), sendo que na operação de divisão há uma proteção para evitar divisão por zero, atribuindo zero ao resultado nesse caso. Também são implementadas operações lógicas como **AND**, **OR** e **NOR**, além das operações de deslocamento lógico à esquerda (**SLL**) e à direita (**SLR**), nas quais o valor de deslocamento é definido pelo sinal **shamt**.

Além das operações que retornam um resultado na saída **result**, a ULA também realiza operações de comparação, utilizadas principalmente nas instruções de desvio condicional. Para isso, a saída **zero** assume valor lógico alto (1) ou baixo (0) dependendo do resultado da comparação entre os operandos, verificando condições de igualdade, diferença, maior, menor ou iguais.

O funcionamento da ULA ocorre de forma combinacional, ou seja, a cada alteração nas entradas, o resultado e a saída de comparação são atualizados imediatamente, sem depender de clock.

2.2.12 Memória de Dados

O módulo **memoriaRAM** implementa a memória de dados de leitura e escrita do processador, que é fundamental para o armazenamento de informações durante a execução do programa. Seguindo a arquitetura Harvard adotada no projeto, esta memória é fisicamente separada da memória de instruções. A sua estrutura foi implementada com base em um template de memória síncrona do Quartus Prime, o que garante uma inferência otimizada dos blocos de memória dedicados (BRAM) disponíveis no FPGA, resultando em uma compilação mais rápida e um design mais eficiente.

A implementação adota um modelo de RAM de porta dupla (*dual-port*), com caminhos independentes para as operações de leitura e escrita. Isso é evidenciado pelas portas distintas para os clocks (**read_clock**, **write_clock**) e para os endereços (**read_addr**, **write_addr**). Ambas as operações são síncronas, o que aumenta a estabilidade do sistema no hardware. A escrita ocorre na borda de subida do **write_clock**, condicionada ao sinal de habilitação **we** (write enable), momento em que o dado da entrada **data** é armazenado na posição indicada por **write_addr**.

De forma análoga, a operação de leitura é síncrona com o **read_clock** e só é executada quando o sinal **memread** está ativo. Quando essas condições são atendidas, o dado armazenado no endereço especificado por **read_addr** é carregado na saída **q** na borda de subida do clock. Esta abordagem totalmente síncrona assegura que o acesso à memória ocorra de forma previsível e estável, alinhada com os ciclos do processador e evitando condições de corrida.

2.2.13 Processador

O módulo `processador` é o componente de nível superior (*top-level*) do projeto, responsável por instanciar e interconectar todos os submódulos desenvolvidos, formando a arquitetura completa do processador MIPS monociclo. Esse módulo implementa o fluxo de controle e de dados, interligando a Unidade de Processamento, a Unidade de Controle, as memórias e os periféricos de entrada e saída. Ele define a interface do processador com o mundo exterior, recebendo sinais de `clock`, `reset`, um botão de avanço (`botaoEnter_n`) e os `switches` de entrada, e fornecendo saídas para os LEDs de status e para os displays de 7 segmentos.

A interatividade do processador é gerenciada por uma lógica de habilitação síncrona. Um sinal interno, `io_stall`, é ativado sempre que a Unidade de Controle decodifica uma instrução de entrada (IN) ou saída (OUT). Com base neste sinal, a lógica `assign pc_enable = !io_stall || (io_stall && pulso_enter);` determina se o processador deve rodar livremente ou pausar. Em instruções normais, `io_stall` é falso e `pc_enable` permanece ativo, permitindo a execução contínua. Ao encontrar uma instrução IN ou OUT, `io_stall` torna-se verdadeiro e `pc_enable` só será ativado por um ciclo quando o usuário pressionar o botão de 'Enter', permitindo assim a execução passo a passo nos momentos de interação. Este sinal `pc_enable` é então utilizado para habilitar a atualização do PC e do `bancoReg`.

O módulo também contém toda a lógica para a interface de visualização. A saída binária do processador é convertida para um formato decimal visível através de um bloco `always_comb`. Esta lógica trata números no intervalo de -999 a 999, converte o valor absoluto para dígitos BCD (*Binary-Coded Decimal*) utilizando operações de divisão e módulo, e controla quatro displays de 7 segmentos. A lógica inclui a exibição do sinal negativo e a supressão de zeros à esquerda para uma visualização mais clara. Uma função local, `decoder_7seg`, é utilizada para traduzir cada dígito BCD para o padrão de 7 segmentos correspondente.

Finalmente, o módulo realiza a instanciação de todos os componentes do datapath, como a ULA, o `bancoReg` e as memórias, e implementa a lógica dos multiplexadores e do cálculo do próximo endereço do PC através de `assigns` e blocos `always_comb`. As conexões entre os módulos seguem o datapath projetado, garantindo a correta propagação dos dados e dos sinais de controle.

2.3 Conjunto de Instruções

Na [Tabela 3](#), apresenta-se o conjunto de instruções adotado no projeto. Para facilitar a compreensão da estrutura de cada tipo, a [Figura 6](#) apresenta o formato geral de cada tipo de instrução, seguindo a mesma estrutura da arquitetura MIPS.

Em relação à [Tabela 3](#), os campos **rd**, **rs** e **rt** representam endereços de 5 *bits* que acessam os 32 registradores disponíveis no banco de registradores. O campo **imediato** corresponde a valores fornecidos diretamente na instrução, sem a necessidade de busca em registradores. O campo **shamt** (*shift amount*) indica a quantidade de deslocamentos a serem realizados em operações de deslocamento. Já o campo **offset** representa um deslocamento relativo utilizado em instruções de desvio, sendo somado ao valor atual do contador de programa (PC). Por fim, o campo **imediato (switches)** refere-se aos valores fornecidos pelas 18 chaves disponíveis no *kit* FPGA, funcionando como entradas binárias diretas para as instruções.

Em seguida, são apresentadas tabelas específicas para cada tipo de instrução: [Tabela 4](#) (tipo R), [Tabela 5](#) (tipo I) e [Tabela 6](#) (tipo J). Nessas tabelas, são especificados os *opcodes* de cada instrução, com base nos formatos ilustrados na figura mencionada.

Figura 6 – Formatos das instruções tipo I, J e R



Fonte: HARTMAN (4)

Tabela 4 – Instruções do tipo R

TIPO R		
OPCODE	FUNCT	INSTRUÇÃO
000000	000000	ADD
000000	000001	SUB
000000	000010	MUL
000000	000011	DIV
000000	000100	AND
000000	000101	OR
000000	000110	NOR
000000	000111	JR
000000	001000	SLL
000000	001001	SLR

Fonte: O Autor

Tabela 3 – Conjunto de instruções do processador

CONJUNTO DE INSTRUÇÕES			
INSTRUÇÃO	OPERAÇÃO	EXPLICAÇÃO	TIPO
ARITMÉTICAS			
ADD	$rd = rs + rt$	Soma	R
ADDI	$rd = rs + \text{imediato}$	Soma com imediato	I
SUB	$rd = rs - rt$	Subtração	R
SUBI	$rd = rs - \text{imediato}$	Subtração com imediato	I
MUL	$rd = rs * rt$	Multiplicação	R
DIV	$rd = rs / rt$	Divisão	R
LÓGICAS			
AND	$rd = rs \& rt$	"E"lógico (bit a bit)	R
ANDI	$rd = rs \& \text{imediato}$	"E"com imediato	I
OR	$rd = rs rt$	"OU"lógico (bit a bit)	R
ORI	$rd = rs \text{imediato}$	"OU"com imediato	I
NOR	$rd = \sim (rs rt)$	Negação do OR	R
DESLOCAMENTO			
SLL	$rd = rt \ll \text{shamt}$	Desloca para esquerda	R
SLR	$rd = rt \gg \text{shamt}$	Desloca para a direita	R
TRANSFERÊNCIA DE DADOS			
LW	$rt = \text{Mem}[rs + \text{offset}]$	Carrega uma palavra	I
SW	$\text{Mem}[rs + \text{offset}] = rt$	Armazena uma palavra	I
DESVIO CONDICIONAL			
BEQ	se $rd == rs$, $PC = PC + \text{offset}$	Desvia se igual	I
BNE	se $rd \neq rs$, $PC = PC + \text{offset}$	Desvia se diferente	I
BLT	se $rd < rs$, $PC = PC + \text{offset}$	Desvia se menor	I
BGT	se $rd > rs$, $PC = PC + \text{offset}$	Desvia se maior	I
BLE	se $rd \leq rs$, $PC = PC + \text{offset}$	Desvia se menor ou igual	I
BGE	se $rd \geq rs$, $PC = PC + \text{offset}$	Desvia se maior ou igual	I
DESVIO INCONDICIONAL			
J	$PC = \text{endereço}$	Pula de endereço	J
JAL	$\text{reg} = PC$; $PC = \text{endereço}$	Chamada de sub-rotina	J
JR	$PC = rs$	Retorno de sub-rotina	R
CONTROLE			
NOP	-	Sem operação	J
HALT	-	Parada	J
ENTRADA E SAÍDA			
IN	$rd = \text{imediato (switches)}$	Sinal de entrada	I
OUT	$\text{saída} = rs$	Sinal de saída	I

Fonte: O Autor

Tabela 5 – Instruções do tipo I

TIPO I	
OPCODE	INSTRUÇÃO
000001	ADDI
000010	SUBI
000011	ANDI
000100	ORI
000101	LW
000110	SW
000111	BEQ
001000	BNE
001001	BLT
001010	BGT
001011	BLE
001100	BGE
001101	IN
001110	OUT

Fonte: O Autor

Tabela 6 – Instruções do tipo J

TIPO J	
OPCODE	INSTRUÇÃO
010000	J
010001	JAL
010010	NOP
010011	HALT

Fonte: O Autor

2.4 Organização da Memória

A arquitetura do processador desenvolvido utiliza o modelo Harvard, no qual as memórias de instruções e de dados são implementadas separadamente, cada uma com barramentos próprios de endereço e dados. Dessa forma, evita-se disputa de barramento entre a busca da instrução e o acesso aos operandos da memória de dados, característica importante em um processador monociclo, onde essas operações podem ocorrer simultaneamente em um mesmo ciclo de execução.

2.4.1 Memória de Instruções

A memória de instruções armazena o programa executado pelo processador em palavras de 32 bits, correspondentes ao tamanho fixo das instruções da arquitetura. Foram implementadas 256 posições de memória, endereçadas por um barramento de 8 bits. O conteúdo da memória é carregado durante a síntese a partir de um arquivo de inicialização contendo o código de máquina previamente montado.

O endereçamento é realizado diretamente pelo registrador PC (Program Counter). Diferentemente da arquitetura MIPS tradicional, em que a memória é endereçada por byte e o PC é incrementado de quatro em quatro, nesta implementação a memória foi organizada em palavras de 32 bits. Assim, cada instrução ocupa exatamente uma posição de memória, permitindo que o PC seja incrementado em apenas uma unidade a cada ciclo ($PC + 1$).

A leitura da memória de instruções é síncrona e ocorre na borda de subida do clock rápido do sistema, correspondente ao clock original da placa antes da divisão de frequência. Com isso, a instrução apontada pelo PC permanece disponível e estabilizada antes da borda do clock lento, utilizado para atualizar os elementos sequenciais do processador, como o PC, o banco de registradores e a memória de dados.

2.4.2 Memória de Dados

A memória de dados também foi implementada com organização em palavras de 32 bits e capacidade de 256 posições endereçadas por 8 bits. Seu acesso é realizado exclusivamente pelas instruções LW (Load Word) e SW (Store Word).

O endereço de acesso não é fornecido diretamente por um registrador. Ele é calculado pela Unidade Lógica e Aritmética (ULA) a partir da soma entre o conteúdo de um registrador base e o deslocamento imediato de 16 bits presente na instrução, após a extensão de sinal. Do resultado obtido, utilizam-se os 8 bits menos significativos para compor o endereço final da memória.

Nesta implementação, leitura e escrita da memória de dados utilizam domínios de clock distintos. A operação de escrita, habilitada pelo sinal de controle **MemWrite** e utilizada pela instrução **SW**, ocorre na borda de subida do clock lento, o mesmo responsável pela sincronização do PC e do banco de registradores.

Já a leitura, habilitada pelo sinal **MemRead** e utilizada pela instrução **LW**, ocorre na borda de subida do clock rápido do sistema. Essa escolha foi adotada para garantir que o dado lido da memória esteja estabilizado antes da borda do clock lento responsável pela escrita no banco de registradores, reduzindo problemas de temporização que poderiam surgir caso ambas as operações fossem realizadas no mesmo domínio de clock.

3 Compilador: Fase de Análise

A fase de análise é a primeira grande etapa do processo de compilação. Sua função é receber o código fonte escrito na linguagem de alto nível, no caso deste projeto a linguagem C-, e verificar se ele está estruturalmente correto, produzindo, ao final, representações intermediárias que serão consumidas pela fase de síntese. De acordo com Aho et al. (1), essa fase é tradicionalmente subdividida em três análises sucessivas: a análise léxica, responsável por agrupar os caracteres do código fonte em tokens; a análise sintática, responsável por verificar se a sequência de tokens obedece à gramática da linguagem; e a análise semântica, responsável por validar regras que vão além da estrutura sintática, como compatibilidade de tipos, declaração prévia de identificadores e regras de escopo.

Neste capítulo apresenta-se, na Seção 3.1, a modelagem do módulo de análise por meio de diagramas SysML de blocos e de atividades. Em seguida, as Seções 3.2, 3.3 e 3.4 descrevem, respectivamente, a implementação de cada uma das três análises citadas.

3.1 Modelagem

A modelagem da fase de análise foi feita utilizando a linguagem SysML (*Systems Modeling Language*), por meio de diagramas de blocos e de atividades. O diagrama de blocos descreve a estrutura estática do módulo, ou seja, os componentes que o compõem, suas responsabilidades e os relacionamentos entre eles. O diagrama de atividades, por sua vez, descreve o comportamento dinâmico do módulo, mostrando o fluxo de execução desde a abertura do arquivo fonte até a produção das estruturas intermediárias.

3.1.1 Diagrama de Blocos

O diagrama de blocos do módulo de análise é apresentado na Figura 7. Ele organiza o módulo em sete blocos principais, divididos em duas categorias: blocos de processamento, que implementam as etapas da análise, e blocos de dados, que armazenam as estruturas geradas ou consultadas ao longo do processo.

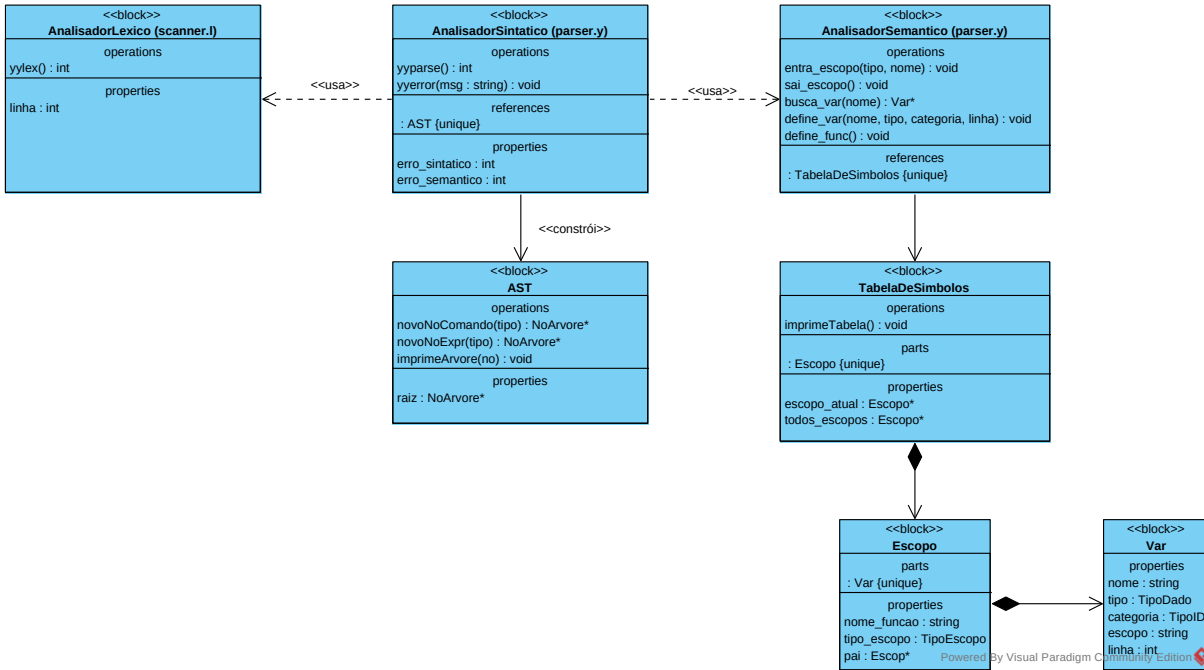


Figura 7 – Diagrama de blocos do módulo de análise.

O bloco **AnalisadorLexico**, implementado no arquivo `scanner.l` com o auxílio da ferramenta Flex, expõe a operação `yylex()`, responsável por retornar o próximo token do arquivo fonte sempre que solicitado. Internamente, ele mantém a propriedade `linha`, utilizada para registrar o número da linha atual do código fonte, informação importante para a emissão de mensagens de erro precisas nas etapas seguintes.

O bloco **AnalisadorSintatico**, implementado no arquivo `parser.y` com o auxílio da ferramenta Bison, expõe as operações `yyparse()`, que dá início ao processo de análise sintática propriamente dito, e `yyerror()`, utilizada para reportar erros sintáticos durante o reconhecimento da gramática. Esse bloco utiliza («usa») o **AnalisadorLexico** para consumir os tokens de entrada e é responsável por construir («constrói») a Árvore Sintática Abstrata (AST). Ele também armazena dois contadores, `erro_sintatico` e `erro_semantico`, que indicam, ao término da análise, se a compilação pode prosseguir para a fase de síntese.

O bloco **AnalisadorSemantico**, embora também esteja implementado no arquivo `parser.y`, foi modelado separadamente para refletir sua responsabilidade distinta. Ele expõe as operações relacionadas à gerência da tabela de símbolos e dos escopos: `entra_escopo()` e `sai_escopo()`, para empilhar e desempilhar escopos à medida que o analisador sintático reconhece blocos e funções; `busca_var()`, para consultar a existência de um identificador nos escopos visíveis; `define_var()` e `define_func()`, para inserir novas declarações de variáveis e funções. Esse bloco mantém uma referência à **TabelaDeSimbolos**, que é a estrutura de dados consultada e atualizada durante toda a

O fluxo principal inicia-se com a abertura do arquivo contendo o código fonte em C-, seguida pela inicialização da Tabela de Símbolos, que cria o escopo global e o deixa pronto para receber as primeiras declarações. Em seguida, a análise é iniciada, transferindo o controle para o analisador sintático.

Na raia do analisador sintático e semântico, o fluxo começa com uma requisição de token ao analisador léxico. O controle é então transferido para a raia da análise léxica, onde a cadeia de caracteres correspondente é lida da entrada. A partir do tipo de lexema reconhecido, o fluxo se ramifica em cinco caminhos possíveis: caracteres irrelevantes, como espaços, tabulações e comentários, são simplesmente ignorados; quebras de linha provocam o incremento do contador `linha`; números e identificadores têm seu valor ou lexema salvo antes da emissão do token correspondente; palavras-chave e símbolos da linguagem retornam imediatamente o token reconhecido; o fim do arquivo provoca a emissão de um token `EOF`; e qualquer cadeia inválida resulta na emissão de um erro léxico. Os caminhos de ignorar caractere e incrementar contador retornam ao início do laço, requisitando novamente a leitura da entrada.

Ao receber um token válido, o analisador sintático verifica se o estado atual da pilha de análise, combinado com o token recebido, satisfaz alguma regra da gramática da linguagem C-. Caso não satisfaça, é emitido um erro sintático. Caso a regra seja reconhecida, executam-se três atividades complementares, modeladas em paralelo: o gerenciamento de escopos e o registro de símbolos na Tabela de Símbolos, a verificação de tipos e demais regras semânticas (com possível emissão de erro semântico) e a instanciação do nó correspondente na AST. Após essas atividades, o fluxo retorna à requisição do próximo token, repetindo-se até que o token `EOF` seja recebido.

Quando o fim do arquivo é alcançado, o controle retorna ao fluxo principal, que realiza a verificação final da existência e da assinatura correta da função `main()`, requisito da linguagem C-. Caso a verificação falhe, é emitido um erro. Em caso de sucesso, a Tabela de Símbolos e a AST são impressas, encerrando a fase de análise e disponibilizando as estruturas intermediárias para a fase de síntese.

3.2 Análise Léxica

A análise léxica é a primeira etapa de processamento do código fonte. Sua função é converter a sequência bruta de caracteres do arquivo de entrada em uma sequência de *tokens*, isto é, unidades léxicas com significado para as etapas seguintes do compilador. No projeto, essa etapa foi implementada com o auxílio da ferramenta Flex, no arquivo `scanner.1`, onde cada padrão da linguagem C- é descrito por uma expressão regular associada a uma ação em C.

O analisador léxico reconhece cinco grandes categorias de elementos: as palavras-

chave da linguagem, que são tratadas como tokens fixos e devem ser reconhecidas antes do padrão geral de identificadores para evitar conflito de prioridade; os símbolos especiais, incluindo operadores aritméticos, relacionais, de atribuição e de pontuação; os identificadores, formados por sequências de letras descritas pela expressão regular `[a-zA-Z]+` com sensibilidade entre maiúsculas e minúsculas; os números inteiros não negativos, descritos por `[0-9]+`; e, por fim, os elementos a serem descartados, como espaços, tabulações, quebras de linha e comentários. A Tabela 7 resume a relação entre lexemas e tokens devolvidos ao analisador sintático.

Tabela 7 – Tokens reconhecidos pelo analisador léxico.

Categoria	Lexemas / Token
Palavras-chave	<code>if</code> , <code>else</code> , <code>int</code> , <code>return</code> , <code>void</code> , <code>while</code>
Operadores aritméticos	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>
Operadores relacionais	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>
Atribuição	<code>=</code>
Pontuação	<code>;</code> , <code>,</code> , <code>(</code> , <code>)</code> , <code>[</code> , <code>]</code> , <code>{</code> , <code>}</code>
Identificadores	ID – padrão <code>[a-zA-Z]+</code>
Números	NUM – padrão <code>[0-9]+</code>

A comunicação entre o analisador léxico e o analisador sintático é realizada por meio da variável global `yylval`, gerada pelo Bison a partir da diretiva `%union`. Quando o scanner reconhece um número, o valor inteiro convertido por `atoi(yytext)` é atribuído ao campo `yylval.val`; quando reconhece um identificador, o lexema é duplicado por `strdup(yytext)` e atribuído ao campo `yylval.id`. Assim, o analisador sintático recebe não apenas a categoria do token, mas também o valor semântico associado a ele, informação que será posteriormente utilizada na construção dos nós da AST e da Tabela de Símbolos.

Para o tratamento de comentários no formato `/* ... */`, foi utilizada uma *start condition* do Flex, denominada `COMENTARIO`, que comuta o scanner para um estado em que todos os caracteres são descartados até a ocorrência da sequência `*/`. Esse mecanismo permite o suporte a comentários com múltiplas linhas, incrementando corretamente o contador de linhas quando quebras de linha ocorrem dentro do bloco comentado. Esse contador, declarado como variável global `linha`, é essencial para que as mensagens de erro emitidas pelas demais fases do compilador apontem com precisão a linha do código fonte em que cada erro foi detectado.

Por fim, qualquer caractere ou sequência que não se enquadre em nenhum dos padrões válidos é capturado pela regra coringa do Flex, que emite uma mensagem de erro léxico contendo o lexema rejeitado e a linha em que foi encontrado, e retorna o token especial `ERROR`. Esse mecanismo garante que entradas malformadas sejam reportadas ao usuário em vez de serem silenciosamente ignoradas, ao mesmo tempo em que permite que o processo de análise prossiga para a busca de possíveis outros erros.

3.3 Análise Sintática

A análise sintática consome a sequência de tokens produzida pelo analisador léxico e verifica se ela é uma sentença válida da linguagem, ou seja, se pode ser derivada a partir de sua gramática. Para a linguagem C-, foi adotada a Gramática Livre de Contexto (GLC) especificada por Louden (5), expressa na Forma de Backus-Naur (BNF). A implementação foi realizada no arquivo `parser.y` com o auxílio da ferramenta Bison, que gera um analisador sintático ascendente do tipo LALR(1) a partir da especificação fornecida.

A gramática utilizada é composta por 29 produções, organizadas hierarquicamente em torno do símbolo inicial `programa`, que representa o ponto de entrada da análise. As produções podem ser agrupadas em três grandes blocos: as declarações, que tratam da definição de variáveis e funções; os comandos (*statements*), que incluem as estruturas de controle de fluxo `if`, `while` e `return`, além de blocos compostos e expressões; e as expressões, que cobrem atribuições, operações aritméticas, comparações relacionais, acessos a vetores e chamadas de função. A gramática completa adotada é apresentada a seguir.

1. $\text{programa} \rightarrow \text{declaração-lista}$
2. $\text{declaração-lista} \rightarrow \text{declaração-lista declaração} \mid \text{declaração}$
3. $\text{declaração} \rightarrow \text{var-declaração} \mid \text{fun-declaração}$
4. $\text{var-declaração} \rightarrow \text{tipo-especificador ID} ; \mid \text{tipo-especificador ID [NUM]} ;$
5. $\text{tipo-especificador} \rightarrow \text{int} \mid \text{void}$
6. $\text{fun-declaração} \rightarrow \text{tipo-especificador ID (params) composto-decl}$
7. $\text{params} \rightarrow \text{param-lista} \mid \text{void}$
8. $\text{param-lista} \rightarrow \text{param-lista , param} \mid \text{param}$
9. $\text{param} \rightarrow \text{tipo-especificador ID} \mid \text{tipo-especificador ID []}$
10. $\text{composto-decl} \rightarrow \{ \text{local-declarações statement-lista} \}$
11. $\text{local-declarações} \rightarrow \text{local-declarações var-declaração} \mid \text{vazio}$
12. $\text{statement-lista} \rightarrow \text{statement-lista statement} \mid \text{vazio}$
13. $\text{statement} \rightarrow \text{expressão-decl} \mid \text{composto-decl} \mid \text{seleção-decl} \mid \text{iteração-decl} \mid \text{retorno-decl}$
14. $\text{expressão-decl} \rightarrow \text{expressão} ; \mid ;$
15. $\text{seleção-decl} \rightarrow \text{if (expressão) statement} \mid \text{if (expressão) statement else statement}$
16. $\text{iteração-decl} \rightarrow \text{while (expressão) statement}$
17. $\text{retorno-decl} \rightarrow \text{return} ; \mid \text{return expressão} ;$

18. $expressão \rightarrow var = expressão \mid simples-expressão$
19. $var \rightarrow ID \mid ID \ [\ expressão \]$
20. $simples-expressão \rightarrow soma-expr \ relational \ soma-expr \mid soma-expr$
21. $relacional \rightarrow <= \mid < \mid > \mid >= \mid == \mid !=$
22. $soma-expr \rightarrow soma-expr \ soma \ termo \mid termo$
23. $soma \rightarrow + \mid -$
24. $termo \rightarrow termo \ mult \ fator \mid fator$
25. $mult \rightarrow * \mid /$
26. $fator \rightarrow (\ expressão \) \mid var \mid ativação \mid NUM$
27. $ativação \rightarrow ID \ (\ args \)$
28. $args \rightarrow arg-lista \mid \text{vazio}$
29. $arg-lista \rightarrow arg-lista \ , \ expressão \mid expressão$

A tradução dessas produções para regras do Bison segue diretamente a hierarquia apresentada, com cada regra associada a uma ou mais ações semânticas escritas em C. Para que essas ações possam manipular valores de tipos heterogêneos, foi utilizada a diretiva `%union`, que define um tipo de dado composto associado a cada não terminal. No projeto, cada não terminal carrega uma estrutura `info`, que contém um ponteiro para um nó da AST (`NoArvore *no`), o tipo semântico do valor produzido (`TipoDado tipo`) e, no caso de listas de argumentos, vetores auxiliares com os tipos e categorias de cada item. Os tokens valorados, por sua vez, são tipados com os campos `val`, para números inteiros, e `id`, para nomes de identificadores.

Dois problemas clássicos do uso direto da gramática merecem destaque por terem exigido tratamento específico no Bison. O primeiro é a ambiguidade de operadores: sem informação adicional, expressões como `a + b * c` poderiam ser interpretadas de mais de uma forma. A solução foi declarar a precedência e a associatividade dos operadores por meio das diretivas `%left` e `%nonassoc`, garantindo que multiplicação e divisão tenham precedência maior que soma e subtração, e que operadores relacionais sejam não associativos, impedindo construções encadeadas como `a < b < c`. O segundo é o conhecido problema do *dangling else*, em que um `else` pode, em princípio, ser associado a mais de um `if` aninhado. Esse conflito foi resolvido com o uso combinado de `%right ELSE` e da diretiva `%prec MENOR` aplicada à regra de seleção sem `else`, fazendo com que o analisador, na presença de ambiguidade, sempre associe o `else` ao `if` mais interno.

À medida que o analisador sintático reduz produções e reconhece estruturas da linguagem, suas ações semânticas vão construindo a Árvore Sintática Abstrata (AST),

estrutura intermediária que será utilizada pelas etapas subsequentes do compilador. Cada nó da árvore é representado pela estrutura `NoArvore`, definida em `globals.h`, que comporta até três filhos, um ponteiro para um nó irmão (utilizado para encadear listas de declarações ou comandos em um mesmo nível hierárquico), um campo `tipo_no` indicando se o nó representa um comando ou uma expressão, e uma união `atr` para armazenar atributos específicos como o nome de um identificador, o valor de uma constante ou o código do operador. Essas instanciações são feitas pelas funções auxiliares `novoNoComando()` e `novoNoExpr()`, implementadas em `util.c`.

O tratamento de erros sintáticos é realizado pela função `yyerror()`, que é invocada automaticamente pelo Bison sempre que a entrada não corresponde a nenhuma derivação válida da gramática. A função reporta a linha em que o erro foi detectado e o token inesperado correspondente, além de ativar uma *flag* global `erro_sintatico`. Essa *flag* é consultada no encerramento da análise para decidir se as fases subsequentes (impressão da AST e geração de código intermediário) devem ou não ser executadas, impedindo que código semanticamente incorreto seja traduzido para a arquitetura alvo.

3.4 Análise Semântica

A análise semântica é responsável por validar aspectos do programa que não podem ser expressos por uma gramática livre de contexto. Enquanto a análise sintática verifica se a sequência de tokens obedece à estrutura da linguagem, a análise semântica verifica se essa estrutura tem significado coerente: se identificadores foram declarados antes do uso, se tipos são compatíveis em operações e atribuições, se chamadas de função respeitam a aridade e os tipos dos parâmetros formais, entre outros aspectos. No projeto, adotou-se a estratégia de tradução dirigida pela sintaxe, em que as verificações semânticas são embutidas nas ações das regras gramaticais do Bison, sendo executadas no mesmo passo da análise sintática.

3.4.1 Tabela de Símbolos

A estrutura central da análise semântica é a Tabela de Símbolos, responsável por armazenar e organizar as informações sobre cada identificador declarado no programa. Sua implementação utiliza listas encadeadas hierárquicas, projetadas para refletir o escopo léxico da linguagem C-. A unidade básica de armazenamento é a estrutura `Var`, que representa um símbolo individual e contém, entre outros campos, o nome do identificador, seu tipo de dado (`T_INT` ou `T_VOID`), sua categoria (`T_VAR`, `T_VETOR` ou `T_FUNC`), o nome do escopo a que pertence, a linha de declaração e, no caso específico de funções, a quantidade, os tipos e as categorias dos parâmetros formais, armazenados em vetores de tamanho `MAX_PARAMS`.

Os símbolos são agrupados em estruturas do tipo **Escopo**, que organizam a tabela hierarquicamente. Cada escopo possui sua própria lista encadeada de variáveis, um ponteiro **pai** para o escopo que o contém, e um campo **tipo_escopo**, que pode assumir três valores: **E_GLOBAL**, para o escopo raiz criado na inicialização; **E_FUNCAO**, instanciado a cada declaração de função; e **E_BLOCO**, gerado dinamicamente para blocos compostos aninhados, como aqueles associados a **if** e **while**. Adicionalmente, cada escopo de função armazena o tipo de retorno esperado, informação consultada pelas verificações associadas ao comando **return**.

Um ponteiro global **escopo_atual** indica o escopo ativo no instante da análise, e é manipulado pelas funções **entra_escopo()** e **sai_escopo()**, que empilham e desempilham escopos à medida que o analisador sintático reconhece blocos. Para fins de impressão final da tabela, todos os escopos criados são também mantidos em uma lista linear, encadeada pelo ponteiro **prox_all**, o que permite percorrer a tabela completa após o término da análise, sem a necessidade de preservar a pilha de escopos.

A busca por identificadores é feita por duas funções complementares: **busca_var_escopo()**, que verifica a presença de um símbolo estritamente no escopo informado, sendo utilizada para detectar declarações duplicadas; e **busca_var()**, que realiza a busca em cadeia, partindo do escopo atual e subindo pelos escopos pais até alcançar o escopo global, sendo utilizada para validar usos de identificadores. Essa busca em cadeia implementa a regra de visibilidade da linguagem e permite o sombreamento (*shadowing*) de variáveis: uma variável local pode ter o mesmo nome de uma global, mas não pode ser redeclarada no mesmo escopo.

3.4.2 Validações Realizadas

As verificações semânticas implementadas cobrem diversos aspectos da linguagem C-. Em declarações de variáveis e parâmetros, o analisador verifica se o tipo declarado é coerente com a categoria do identificador, rejeitando a declaração de variáveis ou parâmetros do tipo **void**, restrito a tipos de retorno de função e à indicação de listas vazias de parâmetros. Para evitar a propagação de erros em cascata, adota-se uma estratégia de recuperação: ao encontrar uma declaração inválida desse tipo, o compilador emite a mensagem de erro e mesmo assim registra o símbolo na tabela com tipo **int**, permitindo que o restante da análise prossiga.

Em usos de identificadores, o analisador consulta a Tabela de Símbolos para verificar se o nome referenciado foi declarado e se sua categoria é compatível com o contexto em que aparece. Tentativas de utilizar uma função como variável escalar, ou de aplicar indexação a um identificador que não é vetor, são detectadas e reportadas. Para o tratamento de vetores, foi introduzido um tipo interno auxiliar denominado **T_VETOR_INT**, que não existe na linguagem C- mas funciona como uma marca lógica: quando o nome de um vetor é

utilizado sem indexação em uma expressão, esse tipo é propagado para os nós superiores, permitindo que operações como soma, comparação ou atribuição direta com vetores sejam barradas.

Em expressões aritméticas, relacionais e atribuições, o analisador verifica a compatibilidade dos tipos dos operandos, rejeitando operações que envolvam valores do tipo `void` ou o uso direto de vetores sem indexação. As estruturas de controle `if` e `while` sofrem verificação análoga em suas expressões condicionais, que devem resultar em um valor escalar inteiro. No comando `return`, o tipo da expressão retornada (ou sua ausência) é confrontado com o tipo de retorno esperado da função corrente, obtido a partir do escopo `E_FUNCAO` que o engloba.

A validação de chamadas de função é feita em três etapas. Inicialmente, verifica-se a existência do identificador e sua pertinência à categoria `T_FUNC`. Em seguida, confronta-se o número de argumentos passados na chamada com o número de parâmetros formais registrados na declaração. Por fim, percorre-se a lista de argumentos verificando, posição a posição, a compatibilidade de tipo e de categoria entre o argumento passado e o parâmetro formal correspondente, distinguindo passagem por valor (escalares) e passagem por referência (vetores). Para viabilizar esse processo durante a análise sintática, foram utilizadas variáveis globais auxiliares (`temp_num_params`, `temp_tipos_params` e `temp_cats_params`) que atuam como *buffers* de acumulação ao longo do reconhecimento da lista de parâmetros da função.

3.4.3 Funções Nativas e Validação da Função `main`

Como o compilador desenvolvido não conta com um processo de *linking* a bibliotecas externas, as funções de entrada e saída foram tratadas como primitivas da linguagem. Antes do início efetivo da análise sintática, a função `inicializar_tabela_simbolos()` cria o escopo global e insere nele as assinaturas das funções `input()`, do tipo `int` e sem parâmetros, e `output(int)`, do tipo `void` e com um único parâmetro inteiro escalar. Com isso, o programador pode invocar essas funções no código fonte como se fizessem parte da linguagem, sem necessidade de declaração explícita.

Após o término bem-sucedido da análise sintática e semântica, a função `valida_main()` é executada para verificar a existência e a conformidade da função `main`. Consulta-se a Tabela de Símbolos para confirmar que o identificador foi declarado no escopo global, que pertence à categoria `T_FUNC`, que possui tipo de retorno `int` ou `void` e que não recebe parâmetros. Caso qualquer um desses requisitos seja descumprido, um erro semântico é emitido, garantindo que apenas programas com um ponto de entrada válido sejam encaminhados para a fase de síntese.

Ao final da análise, todas as informações coletadas estão consolidadas em duas

estruturas: a Tabela de Símbolos, que pode ser impressa com a função `impressao_tabela()` percorrendo a lista linear `lista_escopos`, e a Árvore Sintática Abstrata, cuja raiz é apontada pela variável global `arvoreSintatica`. Caso nenhum erro tenha sido detectado, essas estruturas são utilizadas como entrada para a fase de síntese, descrita no próximo capítulo.

4 Compilador: Fase de Síntese

A fase de síntese é a segunda grande etapa do processo de compilação. Enquanto a fase de análise produz representações intermediárias que descrevem a estrutura e o significado do código fonte, a fase de síntese consome essas representações e as transforma, progressivamente, em código executável pela arquitetura alvo. No projeto, esse processo foi dividido em três etapas sequenciais: a geração do código intermediário em quádruplas, a partir da Árvore Sintática Abstrata; a geração do código assembly da arquitetura MIPS desenvolvida, a partir das quádruplas; e a geração do código executável no formato `.mif` (*Memory Initialization File*), pronto para ser carregado na memória de instruções do processador.

Neste capítulo apresenta-se, na Seção 4.1, a modelagem do módulo de síntese por meio de diagramas SysML. As Seções 4.2, 4.3 e 4.4 descrevem, respectivamente, cada uma das três etapas citadas. Por fim, a Seção 4.5 apresenta o gerenciamento de memória adotado, explicando como variáveis globais, locais, parâmetros e temporários são alocados ao longo da execução de um programa compilado.

4.1 Modelagem

A modelagem do módulo de síntese, assim como a do módulo de análise apresentada no capítulo anterior, foi realizada com a linguagem SysML, por meio de diagramas de blocos e de atividades. Cada uma das etapas do módulo – Gerador de Código Intermediário e Gerador de Código Assembly – foi modelada separadamente, refletindo a divisão adotada na implementação.

4.1.1 Modelagem do Gerador de Código Intermediário

4.1.1.1 Diagrama de Blocos

O diagrama de blocos do GCI é apresentado na Figura 9. Ele organiza o módulo em três blocos principais: o bloco **GeradorCI**, responsável pelas operações de geração propriamente ditas; o bloco **Quad**, que representa a unidade básica de dados produzida pelo gerador; e o bloco **AST**, que representa a estrutura de entrada consumida pelo gerador, herdada da fase de análise.

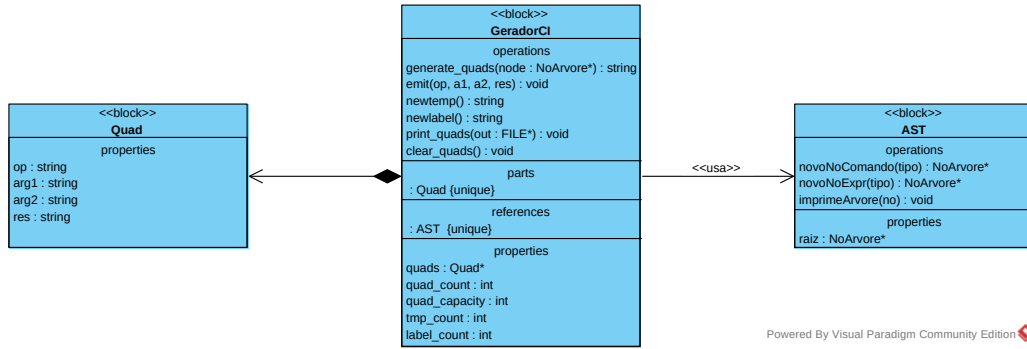


Figura 9 – Diagrama de blocos do Gerador de Código Intermediário.

O bloco **GeradorCI** expõe um conjunto de operações que formam a interface pública do módulo. A operação **generate_quads()** é a função central, sendo invocada pelo fluxo principal com o nó raiz da AST e responsável por percorrer recursivamente a árvore, emitindo as quádruplas correspondentes a cada construção encontrada. A operação **emit()** é o utilitário básico de inserção: recebe um operador e até três operandos, aloca uma nova posição no vetor dinâmico de quádruplas e a preenche. As operações **newtemp()** e **newlabel()** produzem, respectivamente, nomes únicos para registradores temporários (no formato **\$t0**, **\$t1**, ...) e para rótulos de salto (no formato **L0**, **L1**, ...), utilizados na tradução de expressões e estruturas de controle. Por fim, **print_quads()** imprime a sequência completa de quádruplas geradas, e **clear_quads()** libera a memória alocada ao final do processo.

Internamente, o bloco **GeradorCI** mantém quatro propriedades de estado: o vetor **quads**, que armazena todas as quádruplas geradas; **quad_count** e **quad_capacity**, que controlam, respectivamente, o número atual de quádruplas e a capacidade do vetor (redimensionado dinamicamente sempre que necessário); e os contadores **tmp_count** e **label_count**, utilizados pelas funções **newtemp()** e **newlabel()** para garantir a unicidade dos nomes gerados.

O bloco **Quad** representa a estrutura de uma única quádrupla, composta por quatro campos do tipo *string*: **op**, que armazena o operador da instrução; **arg1** e **arg2**, que armazenam os operandos; e **res**, que armazena o destino do resultado. O relacionamento de composição («part») entre **GeradorCI** e **Quad** indica que as quádruplas só existem enquanto o gerador estiver ativo; ao final do processo, todas são liberadas. A relação de uso («usa») entre **GeradorCI** e **AST** indica que o gerador apenas consulta a árvore, sem modificá-la.

4.1.2 Diagrama de Atividades

O diagrama de atividades do GCI é apresentado na Figura 10. Ele descreve o fluxo de execução desde a invocação inicial até a emissão completa do código intermediário,

organizado em duas raiais: o fluxo principal e o gerador propriamente dito.

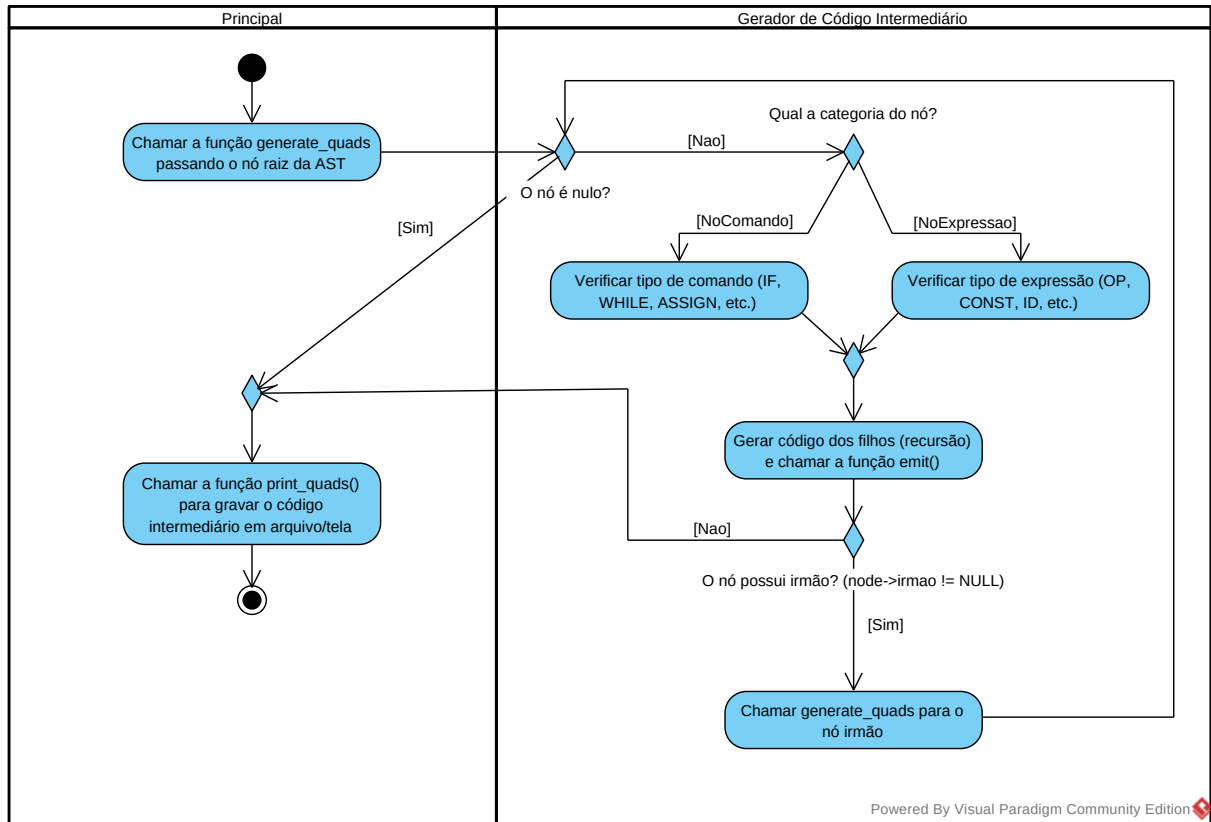


Figura 10 – Diagrama de atividades do Gerador de Código Intermediário.

O fluxo principal inicia-se com a invocação da função `generate_quads()` passando como argumento o nó raiz da AST. A partir desse ponto, o controle é transferido para a raia do gerador, que realiza um percurso recursivo pela árvore. O primeiro teste verifica se o nó atual é nulo; em caso afirmativo, encerra-se a recursão e o fluxo retorna ao principal, que executa a impressão das quádruplas geradas. Caso contrário, o gerador consulta a categoria do nó por meio do campo `tipo_no`: se for um nó de comando, identifica-se o tipo específico (`comandoIF`, `comandoRepeat`, `comandoAssign`, `comandoReturn` ou `comandoCompound`); se for um nó de expressão, identifica-se o tipo correspondente (`OpAritmetica`, `Constante`, `Identificador`, `ChamadaFunc`, `TipoEspec` ou `Vetor`).

Independentemente da categoria, o tratamento prossegue com a geração recursiva do código dos filhos do nó e a chamada de `emit()` para registrar as quádruplas associadas à construção reconhecida. Em seguida, verifica-se se o nó atual possui um nó irmão; caso possua, a função `generate_quads()` é invocada novamente sobre esse nó, garantindo o tratamento sequencial de comandos no mesmo nível hierárquico, como as declarações em uma lista ou os comandos em um bloco. Caso não haja irmão, o fluxo retorna ao chamador recursivo. Esse procedimento se repete até que toda a árvore tenha sido percorrida.

4.1.3 Modelagem do Gerador de Código Assembly

4.1.3.1 Diagrama de Blocos

O diagrama de blocos do Gerador de Assembly é apresentado na Figura 11. Diferentemente do GCI, cuja entrada é uma única estrutura hierárquica (a AST), o Gerador de Assembly consome duas estruturas distintas produzidas pelas etapas anteriores – a Tabela de Símbolos e o vetor de quádruplas – e mantém internamente duas coleções auxiliares específicas do processo de tradução para a arquitetura alvo.

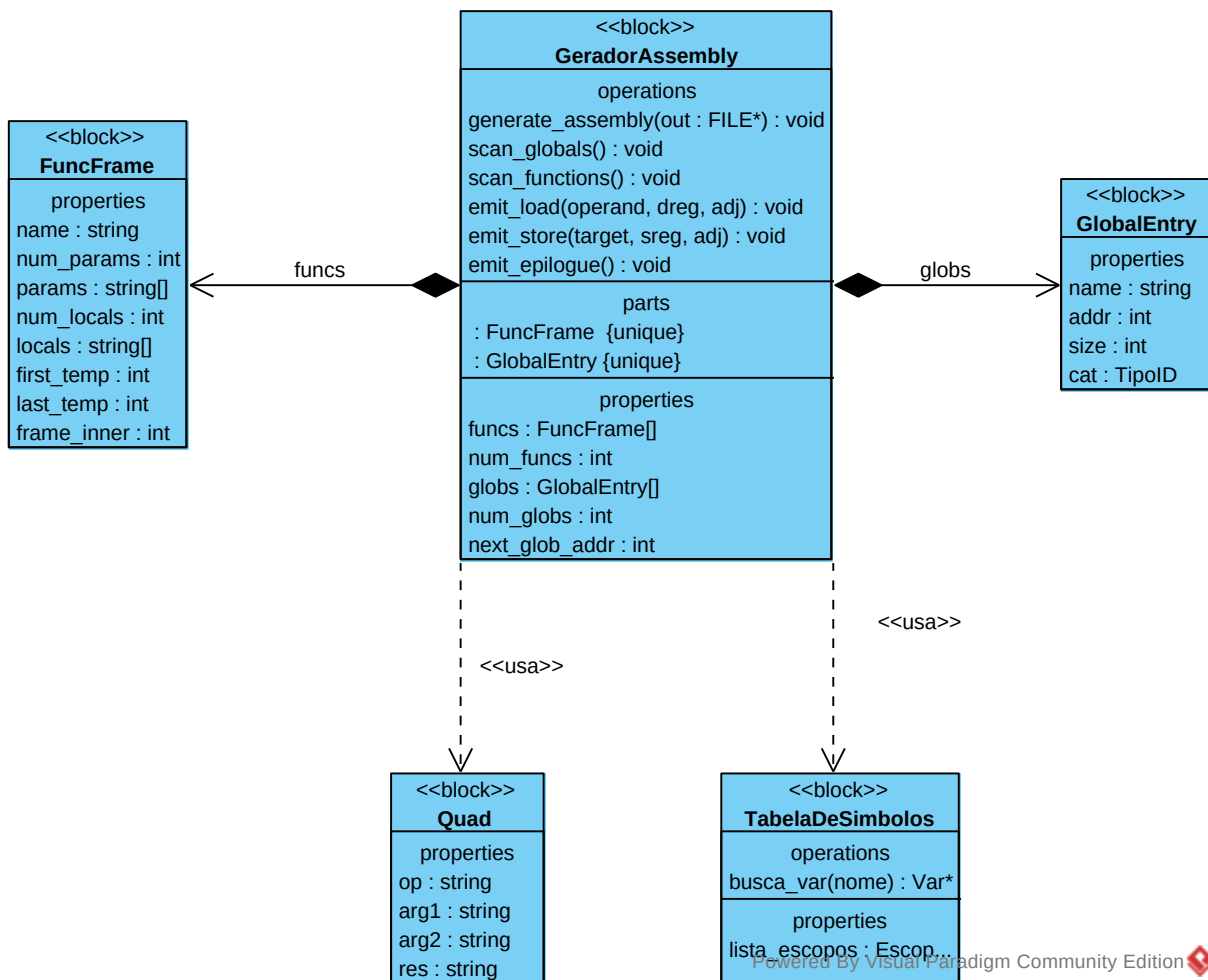


Figura 11 – Diagrama de blocos do Gerador de Código Assembly.

O bloco central `GeradorAssembly`, implementado no arquivo `cgen.c`, expõe a operação pública `generate_assembly()`, que orquestra todo o processo de tradução e escreve o código produzido no arquivo de saída. As operações `scan_globals()` e `scan_functions()` constituem a primeira passagem do gerador, responsáveis por coletar informações estruturais sobre variáveis globais e sobre cada função declarada no programa. As operações `emit_load()`, `emit_store()` e `emit_epilogue()` são utilitários da segunda passagem,

encarregados de encapsular a lógica de acesso à memória e de finalização de função, cujo detalhamento pode ser encontrado na Seção 4.3.

O bloco **GeradorAssembly** mantém propriedades que refletem essas duas coleções auxiliares: **funcs**, que armazena os quadros de ativação de cada função identificada, e **globals**, que armazena o mapeamento das variáveis globais para endereços absolutos, junto com o contador **next_glob_addr** que controla a próxima posição disponível na região global da memória de dados. Os relacionamentos de composição («part») que ligam **GeradorAssembly** aos blocos **FuncFrame** e **GlobalEntry** indicam que essas coleções são internas ao gerador e existem apenas durante o processo de tradução.

O bloco **FuncFrame** representa o registro de ativação (*frame*) de uma função, contendo o nome da função, a lista de parâmetros formais, a lista de variáveis locais e o intervalo de temporários utilizados. Essas informações são fundamentais para o cálculo dos *offsets* de acesso à pilha durante a segunda passagem: cada acesso a um parâmetro, variável local ou temporário no assembly gerado é traduzido em uma instrução LW ou SW com *offset* calculado a partir dessas listas. O bloco **GlobalEntry**, por sua vez, armazena o nome, o endereço absoluto e o tamanho de cada variável global, permitindo que acessos a essas variáveis sejam traduzidos em instruções com endereçamento baseado no registrador **\$zero**.

Por fim, os blocos **Quad** e **TabelaDeSimbolos**, ligados ao **GeradorAssembly** por relacionamentos de uso («usa»), representam as estruturas externas consumidas pelo gerador. O bloco **Quad** é o mesmo apresentado no diagrama do GCI, agora atuando como entrada da próxima etapa; e o bloco **TabelaDeSimbolos**, produzido durante a fase de análise, é consultado principalmente pela função **sym_is_array()** para determinar se um identificador corresponde a um vetor, informação essencial para o tratamento correto da passagem de argumentos em chamadas de função.

4.1.3.2 Diagrama de Atividades

O diagrama de atividades do Gerador de Assembly é apresentado na Figura 12. Ele descreve o fluxo de execução em duas raias: o fluxo principal, responsável por iniciar o processo e finalizar a escrita do arquivo, e o gerador propriamente dito, que realiza as duas passagens sobre as estruturas de entrada.

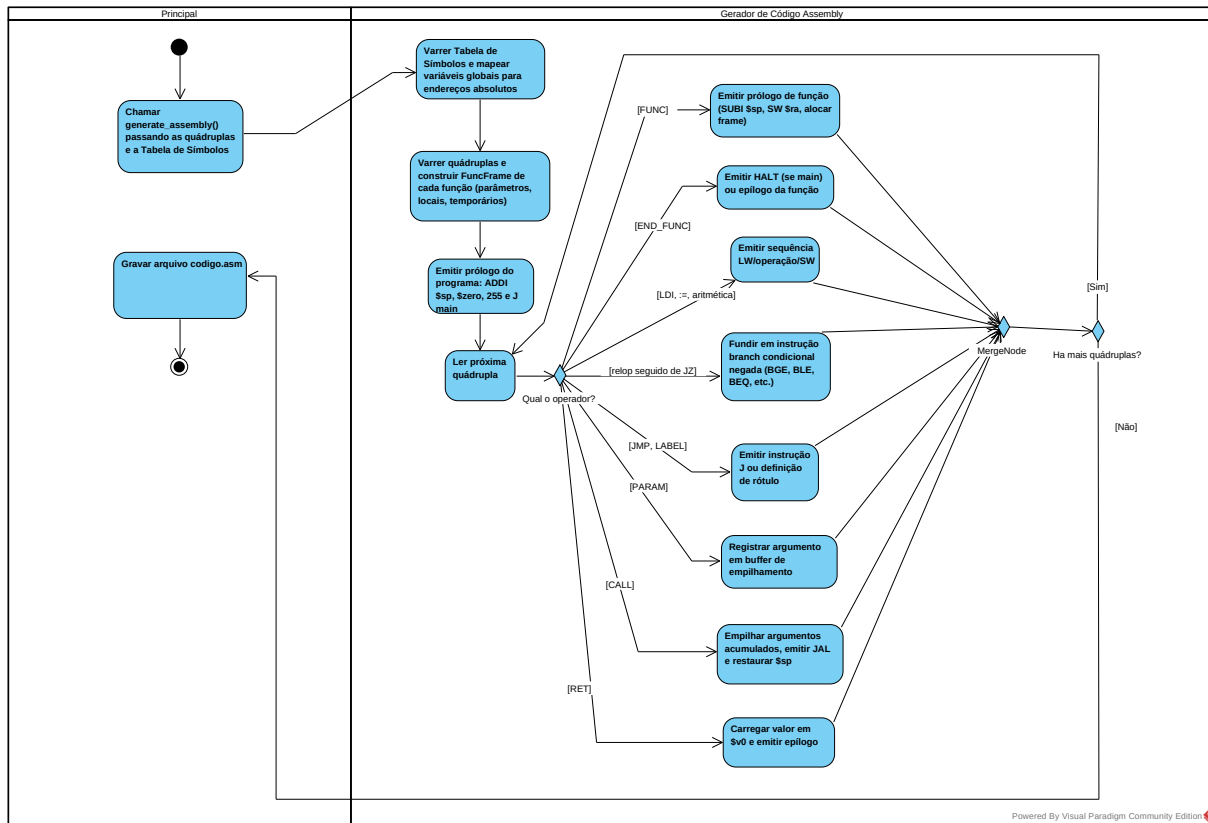


Figura 12 – Diagrama de atividades do Gerador de Código Assembly.

O fluxo inicia-se com a invocação de `generate_assembly()` pelo fluxo principal, passando como entrada o vetor de quádruplas produzido pelo GCI e a Tabela de Símbolos construída na fase de análise. O controle é então transferido para o gerador, que executa a primeira passagem em duas etapas sequenciais: a varredura da Tabela de Símbolos para mapear cada variável global a um endereço absoluto na região estática da memória de dados, e a varredura do vetor de quádruplas para identificar os limites de cada função (delimitada por `FUNC/END_FUNC`) e construir seu respectivo `FuncFrame`, incluindo parâmetros, variáveis locais e o intervalo de temporários utilizados.

Concluída a fase preparatória, o gerador emite o prólogo do programa, composto pelas instruções de inicialização do ponteiro de pilha (`ADDI $sp, $zero, 255`) e do salto incondicional para a função `main` (`J main`). A partir desse ponto, inicia-se o laço principal da segunda passagem, no qual cada quádrupla é lida sequencialmente e submetida a um teste que ramifica o tratamento conforme o operador identificado.

O ramo correspondente a `FUNC` emite o prólogo da função (`SUBI $sp, SW $ra` e alocação do *frame* interno); o ramo de `END_FUNC` emite a instrução `HALT`, caso a função encerrada seja a `main`, ou o epílogo padrão de retorno, para as demais funções. Operações aritméticas, atribuições e cargas de constante são traduzidas em sequências no padrão *load-compute-store*, utilizando os registradores de rascunho `$t0`, `$t1` e `$t2`. Operações

relacionais são tratadas em conjunto com o JZ imediatamente subsequente, fundindo as duas quádruplas em uma única instrução *branch* da arquitetura com o operador relacional negado, conforme descrito na Seção 4.3. Instruções JMP e LABEL são traduzidas diretamente em J e definições de rótulo. Quádruplas PARAM não emitem código imediatamente, mas registram seus argumentos em um *buffer* interno para processamento posterior; quando uma quádrupla CALL é encontrada, todos os argumentos acumulados são empilhados de uma só vez, a instrução JAL é emitida, e o ponteiro de pilha é restaurado após o retorno. Por fim, quádruplas RET carregam o valor de retorno no registrador \$v0 e emitem o epílogo da função corrente.

Após o tratamento de cada quádrupla, verifica-se se ainda há instruções a processar; em caso afirmativo, o laço reinicia com a leitura da próxima quádrupla; em caso negativo, o controle retorna ao fluxo principal, que finaliza a escrita do arquivo `codigo.asm` e encerra o processo.

4.1.4 Modelagem do Gerador de Código Binário

4.1.4.1 Diagrama de Blocos

O diagrama de blocos do Gerador de Código Binário é apresentado na Figura 13. Trata-se do módulo mais simples do conjunto da fase de síntese, tanto do ponto de vista estrutural quanto do ponto de vista das dependências externas: seu único consumo é o arquivo textual de assembly produzido pela etapa anterior, e sua única saída é o arquivo binário no formato `.mif`.

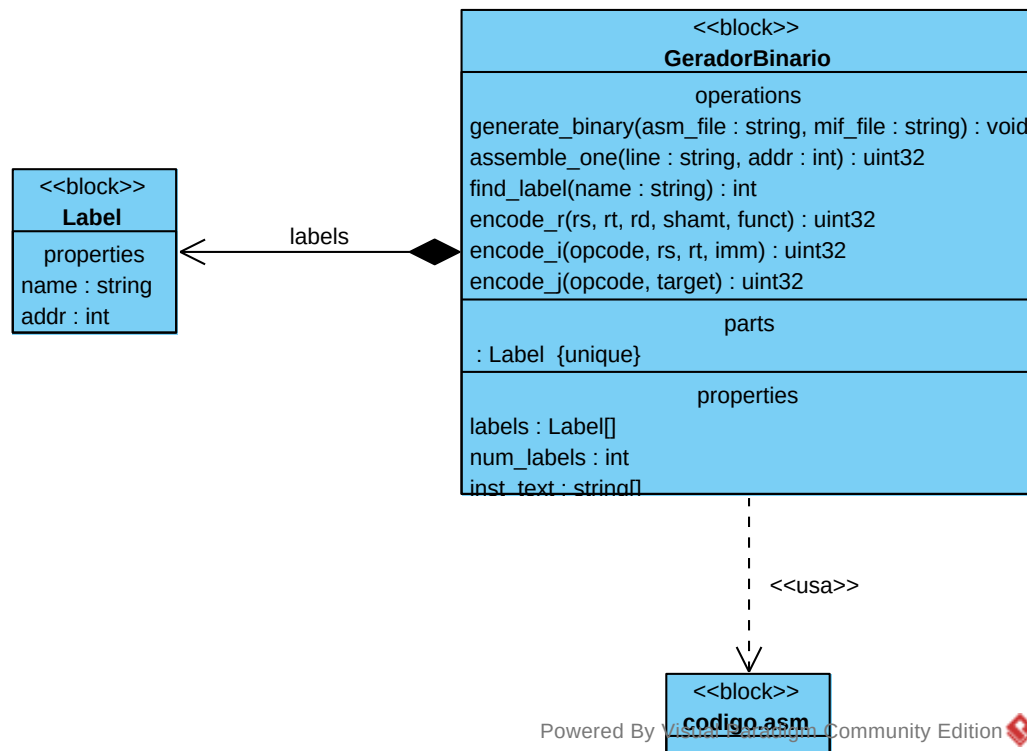


Figura 13 – Diagrama de blocos do Gerador de Código Binário.

O bloco central **GeradorBinario**, implementado no arquivo `assembler.c`, expõe como operação pública a função `generate_binary()`, que recebe o caminho do arquivo assembly de entrada e o caminho do arquivo binário de saída, orquestrando todo o processo de montagem. Como operações internas, o bloco expõe a função `assemble_one()`, responsável pela codificação de uma única instrução assembly em sua palavra binária correspondente; a função `find_label()`, que consulta a tabela de rótulos coletados na primeira passagem; e as três funções especializadas `encode_r()`, `encode_i()` e `encode_j()`, cada uma responsável por compor a palavra binária de 32 bits conforme o formato de instrução correspondente, aplicando os deslocamentos de bit e as máscaras necessárias para cada campo.

Como propriedades de estado, o **GeradorBinario** mantém duas coleções: o vetor `labels`, que armazena os rótulos e seus endereços, e o vetor `inst_text`, que armazena o texto de cada instrução lida na primeira passagem para posterior codificação. Os contadores `num_labels` e `num_insts` controlam o número de elementos ativos em cada uma dessas coleções.

O bloco **Label** representa a estrutura elementar da tabela de rótulos, contendo apenas dois campos: o nome do rótulo (`name`) e o endereço da instrução à qual ele foi associado durante a primeira passagem (`addr`). O relacionamento de composição («**part**») entre **GeradorBinario** e **Label** indica que essa tabela é interna ao gerador e existe apenas durante o processo de montagem.

Por fim, o bloco `codigo.asm`, ligado ao `GeradorBinario` por um relacionamento de uso («usa»), representa o artefato de entrada consumido pelo módulo. Diferentemente dos blocos anteriores, ele não representa uma estrutura de dados em memória, mas sim um arquivo em disco produzido pela etapa anterior do pipeline; sua presença explícita no diagrama serve para deixar clara a dependência de fluxo entre os módulos da fase de síntese.

4.1.4.2 Diagrama de Atividades

O diagrama de atividades do Gerador de Código Binário é apresentado na Figura 14. Ele descreve o fluxo em duas raias – o fluxo principal e o gerador propriamente dito – e explicita a estratégia de duas passagens sequenciais, necessária para viabilizar a resolução correta de rótulos que podem ser referenciados antes de serem definidos no código assembly.

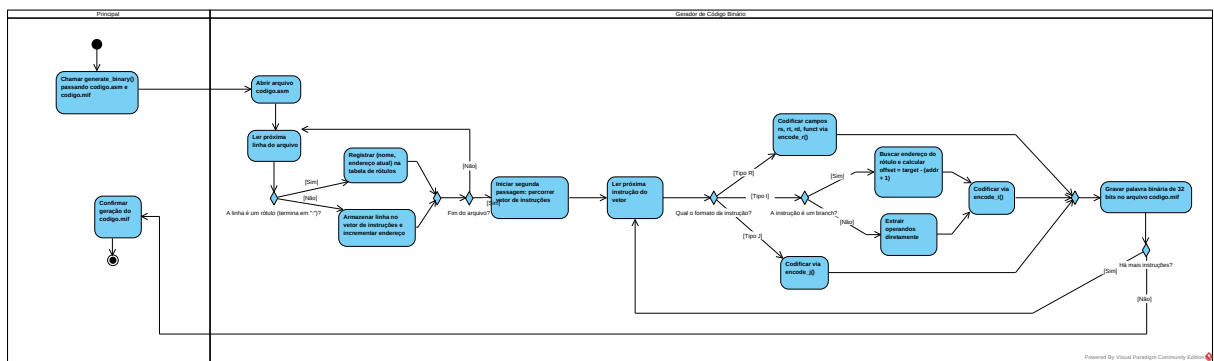


Figura 14 – Diagrama de atividades do Gerador de Código Binário.

O fluxo inicia-se com a invocação de `generate_binary()` pelo fluxo principal, indicando o arquivo assembly de entrada e o arquivo binário de saída. O controle é transferido para o gerador, que abre o arquivo `codigo.asm` e inicia a primeira passagem.

Na primeira passagem, cada linha do arquivo é lida sequencialmente e submetida a um teste: se a linha termina com o caractere “:”, é interpretada como a definição de um rótulo, e o par (nome, endereço da próxima instrução) é registrado na tabela interna de rótulos; caso contrário, a linha é interpretada como uma instrução assembly e armazenada no vetor de instruções, com o contador de endereços incrementado em uma unidade. Esse laço se repete até o fim do arquivo, ao final do qual dispõe-se de duas estruturas completas: a tabela de rótulos, com todas as resoluções simbólicas necessárias, e o vetor de instruções, pronto para ser processado.

A segunda passagem consiste no laço principal de codificação. Cada instrução do vetor é lida sequencialmente, e seu mnemônico é utilizado para determinar o formato de instrução aplicável: tipo R, para operações lógicas e aritméticas entre registradores; tipo I, para operações imediatas, acessos à memória e desvios condicionais; ou tipo J, para

desvios incondicionais e instruções de controle. A partir do formato identificado, uma das três funções auxiliares `encode_r()`, `encode_i()` ou `encode_j()` é chamada para compor a palavra binária de 32 bits correspondente. No caso específico de desvios condicionais, é necessário consultar previamente a tabela de rótulos para obter o endereço absoluto do alvo e, a partir dele, calcular o *offset* relativo conforme a fórmula apresentada na Seção 4.4. Após a codificação, a palavra binária resultante é gravada no arquivo `codigo.mif`, precedida de um comentário com o endereço da instrução e seu texto original para fins de depuração.

O laço se encerra quando todas as instruções do vetor tiverem sido codificadas, momento em que o controle retorna ao fluxo principal, que confirma a geração bem-sucedida do arquivo binário e encerra o processo. Como resultado, obtém-se um arquivo pronto para ser utilizado tanto em simulações em SystemVerilog, por meio da função `$readmemb()`, quanto em implementações reais em FPGA por meio da ferramenta Quartus.

4.2 Geração do Código Intermediário

A geração de código intermediário recebe como entrada a Árvore Sintática Abstrata construída na fase de análise e produz como saída uma sequência linear de instruções no formato de Código de Três Endereços (*Three-Address Code*), implementado por meio de quádruplas. Conforme apresentado por Aho et al. (1), essa forma intermediária tem por vantagem a proximidade com o código de máquina, ao mesmo tempo em que permanece independente da arquitetura alvo, o que facilita tanto a tradução posterior para assembly quanto a aplicação de eventuais técnicas de otimização independentes de máquina.

Cada quádrupla é representada pela estrutura `Quad`, composta por quatro campos do tipo *string*: o operador (`op`), dois argumentos (`arg1` e `arg2`) e o destino do resultado (`res`). Campos não utilizados em uma instrução específica são preenchidos com o caractere `-`, garantindo regularidade na impressão e no processamento posterior. O conjunto completo de operadores definidos para o projeto é apresentado na Tabela 8, agrupando operações de atribuição, aritmética, comparação relacional, controle de fluxo e gerenciamento de funções.

Tabela 8 – Tipos de quádruplas definidos no projeto.

Nome	Op	Arg1	Arg2	Resultado
Atribuição	<code>:=</code>	origem (var/temp)	–	destino
Soma	<code>+</code>	operando1	operando2	temporário
Subtração	<code>-</code>	operando1	operando2	temporário
Multiplicação	<code>*</code>	operando1	operando2	temporário
Divisão	<code>/</code>	operando1	operando2	temporário
Menor que	<code><</code>	operando1	operando2	temporário
Menor ou igual	<code><=</code>	operando1	operando2	temporário
Maior que	<code>></code>	operando1	operando2	temporário
Maior ou igual	<code>>=</code>	operando1	operando2	temporário
Igualdade	<code>==</code>	operando1	operando2	temporário
Diferença	<code>!=</code>	operando1	operando2	temporário
Carga de constante	<code>LDI</code>	constante	–	temporário
Leitura de vetor	<code>LDV</code>	<code>v[\$ti]</code>	–	temporário
Definição de <i>label</i>	<code>LABEL</code>	–	–	<i>label</i>
Desvio condicional	<code>JZ</code>	condição	–	<i>label</i>
Desvio incondicional	<code>JMP</code>	–	–	<i>label</i>
Início de função	<code>FUNC</code>	nome da função	–	–
Fim de função	<code>END_FUNC</code>	nome da função	–	–
Parâmetro formal	<code>PARAM_DECL</code>	nome do parâmetro	–	–
Passagem de argumento	<code>PARAM</code>	valor passado	–	–
Chamada de função	<code>CALL</code>	nome da função	–	temporário
Retorno	<code>RET</code>	valor retornado	–	–

A infraestrutura de armazenamento foi implementada no arquivo `util.c` sobre um vetor dinâmico de estruturas `Quad`. A função `emit()` é responsável por inserir uma nova quádrupla no vetor: ela verifica se há espaço disponível e, caso contrário, dobra a capacidade do vetor por meio de `realloc()`, garantindo crescimento eficiente sem necessidade de prever o tamanho final do código. Cada um dos quatro campos da quádrupla recebe uma cópia segura da *string* de entrada, produzida pela função `strdup_safe()`, o que evita problemas de *dangling pointers* caso o chamador libere as *strings* originais. Os contadores `tmp_count` e `label_count` são utilizados pelas funções auxiliares `newtemp()` e `newlabel()` para gerar nomes únicos no formato `$tn` e `Ln`, respectivamente.

A função central do gerador, `generate_quads()`, recebe um nó da AST e retorna uma *string* que indica o local onde o valor produzido pelo nó foi armazenado, no caso de expressões, ou `NULL`, no caso de comandos. Internamente, a função opera por casos, ramificando o tratamento de acordo com o tipo do nó. Para nós de expressão, constantes são carregadas em um novo temporário por meio da instrução `LDI`; identificadores são retornados pelo próprio nome, permitindo que sejam utilizados diretamente como operandos pelas operações superiores; operações aritméticas e relacionais geram código para os dois filhos, criam um novo temporário e emitem a quádrupla correspondente; acessos a vetor avaliam primeiro a expressão do índice e emitem uma instrução `LDV`; e chamadas de função geram código para cada argumento, emitem uma instrução `PARAM` para cada um deles e,

ao final, emitem a instrução **CALL** que registra o retorno em um novo temporário.

O tratamento de comandos é o que efetivamente lineariza o fluxo de controle do programa. Para uma atribuição, gera-se primeiro o código da expressão do lado direito e, em seguida, emite-se uma quádrupla $:=$ que copia o resultado para o destino, com tratamento diferenciado caso o destino seja um elemento de vetor. Para um comando **if**, criam-se dois rótulos, um para o caso falso e outro para o final do comando: avalia-se a condição, emite-se uma instrução **JZ** (*Jump if Zero*) para o rótulo do caso falso, gera-se o código do bloco **then**, emite-se um **JMP** para o rótulo final, define-se o rótulo do caso falso, gera-se o código do bloco **else** e, por fim, define-se o rótulo final. Para um comando **while**, define-se um rótulo no início do laço, avalia-se a condição, emite-se um **JZ** para um rótulo de saída, gera-se o código do corpo, emite-se um **JMP** de volta ao início e define-se o rótulo de saída. Funções são delimitadas pelas quádruplas **FUNC** e **END_FUNC**, e seus parâmetros formais são registrados explicitamente por instruções **PARAM_DECL**.

A título de exemplo, considere-se o programa apresentado a seguir, que lê dois valores inteiros e imprime o maior deles:

```

1 void main(void) {
2     int x;
3     int y;
4     x = input();
5     y = input();
6     if (x > y)
7         output(x);
8     else
9         output(y);
10 }
```

A aplicação do gerador sobre a AST construída para esse programa resulta no código intermediário apresentado a seguir, em que se observa a delimitação da função por **FUNC/END_FUNC**, a tradução das chamadas a **input()** e **output()** para sequências de **CALL** e **PARAM**, e a tradução da estrutura **if-else** para a combinação de instruções **>**, **JZ**, **JMP** e **LABEL**.

```

1 0: (FUNC, main, -, -)
2 1: (CALL, input, -, $t0)
3 2: (:=, $t0, -, x)
4 3: (CALL, input, -, $t1)
5 4: (:=, $t1, -, y)
6 5: (>, x, y, $t2)
7 6: (JZ, $t2, -, L0)
8 7: (PARAM, x, -, -)
9 8: (CALL, output, -, $t3)
```

```
10 9: (JMP, -, -, L1)
11 10: (LABEL, -, -, L0)
12 11: (PARAM, y, -, -)
13 12: (CALL, output, -, $t4)
14 13: (LABEL, -, -, L1)
15 14: (END_FUNC, main, -, -)
```

4.3 Geração do Código Assembly

A etapa seguinte da fase de síntese transforma o código intermediário em código assembly para a arquitetura MIPS monociclo desenvolvida na disciplina anterior. Essa tradução foi implementada no arquivo `cgen.c`, em uma estratégia em duas passagens. A primeira passagem varre a Tabela de Símbolos e o vetor de quádruplas para coletar informações estruturais sobre as variáveis globais e sobre cada função do programa; a segunda passagem percorre as quádruplas uma a uma, emitindo, para cada uma, a sequência de instruções assembly correspondente, com base nas informações coletadas na primeira passagem.

Na primeira passagem, a função `scan_globals()` percorre todos os escopos da Tabela de Símbolos por meio do ponteiro `lista_escopos` e identifica as variáveis e vetores declarados no escopo global, atribuindo a cada um deles um endereço absoluto na memória de dados, a partir do endereço zero. Em seguida, `scan_functions()` percorre o vetor de quádruplas identificando blocos delimitados por `FUNC/END_FUNC` e, para cada função, registra uma estrutura `FuncFrame` contendo o nome, a lista de parâmetros, a lista de variáveis locais e os limites do intervalo de temporários utilizados. Essas informações são essenciais para o cálculo dos *offsets* de memória empregados na segunda passagem.

A segunda passagem é executada pela função `generate_assembly()`, que percorre o vetor de quádruplas e atua por casos sobre o operador de cada uma. As quádruplas `LDI`, `:=`, `+`, `-`, `*` e `/` são traduzidas para sequências que carregam os operandos em registradores temporários do processador (`$t0`, `$t1`), executam a operação aritmética correspondente (`ADDI`, `ADD`, `SUB`, `MUL` ou `DIV`) e armazenam o resultado de volta na memória, na posição correspondente ao destino da quádrupla. As funções auxiliares `emit_load()` e `emit_store()` encapsulam a lógica de cálculo de endereços, distinguindo automaticamente entre acessos a temporários, a variáveis locais, a parâmetros e a variáveis globais.

As quádruplas de comparação relacional (`<`, `<=`, `>`, `>=`, `==` e `!=`) recebem um tratamento especial: como o conjunto de instruções da arquitetura inclui instruções de desvio condicional do tipo *branch-if-relop* (`BLT`, `BLE`, `BGT`, `BGE`, `BEQ` e `BNE`), o gerador detecta o padrão recorrente `<relop>` seguido imediatamente por `JZ` e funde as duas quádruplas em uma única instrução *branch* cuja condição é a negação do operador relacional original.

Por exemplo, a sequência intermediária “ $x > y$; JZ \$t2 L0” é traduzida para a instrução “BLE \$t0, \$t1, L0”, uma vez que o desvio para L0 ocorre exatamente quando a condição $x > y$ é falsa. Esse *merge* reduz o número de instruções emitidas e elimina a necessidade de calcular explicitamente o valor booleano da comparação.

As quádruplas de controle de fluxo JMP e LABEL são traduzidas diretamente para instruções J e definições de rótulo no assembly. O tratamento de chamadas de função, por outro lado, exige cuidados particulares. A estratégia adotada é a de empilhamento diferido: as quádruplas PARAM não emitem instruções imediatamente, mas apenas registram os argumentos em um vetor auxiliar interno (*deferred_args*). Quando a quádrupla CALL é processada, o gerador pré-decrementa o ponteiro de pilha \$sp pelo número total de argumentos e os empilha todos de uma vez, na ordem inversa, garantindo que os *offsets* dos demais operandos do *frame* permaneçam consistentes durante a sequência de empilhamento. Em seguida, emite-se a instrução JAL, que salta para a função chamada armazenando o endereço de retorno em \$ra. Após o retorno, \$sp é restaurado e, se houver, o valor de retorno (em \$v0) é armazenado no destino da chamada.

As funções nativas `input()` e `output()` recebem tratamento especial, uma vez que correspondem diretamente a instruções de entrada e saída do processador. A chamada a `input()` é traduzida para uma instrução IN \$v0 seguida da escrita do valor lido no destino indicado, e a chamada a `output()` é traduzida para uma instrução OUT \$t0 sobre o valor passado como argumento, sem necessidade de empilhamento.

Para ilustrar a saída do gerador, apresenta-se a seguir o código assembly produzido a partir do código intermediário do exemplo anterior. Observa-se a presença do prólogo (com inicialização de \$sp em 255 e salto para `main`), o salvamento de \$ra, a reserva de espaço na pilha para variáveis locais e temporários e o tratamento das instruções de entrada, saída e desvio condicional descritas acima.

```
1      ADDI $sp, $zero, 255
2      J main
3
4 main:
5      SUBI $sp, $sp, 1
6      SW $ra, 0($sp)
7      SUBI $sp, $sp, 7
8      IN $v0
9      SW $v0, 0($sp)
10     LW $t0, 0($sp)
11     SW $t0, 5($sp)
12     IN $v0
13     SW $v0, 1($sp)
14     LW $t0, 1($sp)
```



```
15      SW $t0, 6($sp)
16      LW $t0, 5($sp)
17      LW $t1, 6($sp)
18      BLE $t0, $t1, L0
19      LW $t0, 5($sp)
20      OUT $t0
21      J L1
22 L0:
23      LW $t0, 6($sp)
24      OUT $t0
25 L1:
26      HALT
```

A correspondência entre o código intermediário e o assembly gerado pode ser verificada diretamente: a fusão das quádruplas 5 e 6 (> e JZ) em uma única instrução BLE, a tradução do JMP L1 em J L1, a definição dos rótulos L0 e L1 no assembly e a substituição do END_FUNC main pela instrução HALT, que encerra a execução do processador. Os *offsets* 5(\$sp) e 6(\$sp) correspondem, respectivamente, às variáveis locais x e y, posicionadas após os temporários no *frame* da função, conforme descrito na Seção 4.5.

4.4 Geração do Código Executável

A última etapa da fase de síntese é a montagem (*assembly*) propriamente dita: a tradução do código assembly textual em uma sequência de palavras binárias de 32 bits, no formato *.mif* (*Memory Initialization File*). Esse arquivo é diretamente reconhecido por simuladores SystemVerilog através da função `$readmemb()` e pode ser carregado na memória de instruções do processador implementado em FPGA por meio da ferramenta Quartus. Cada linha do arquivo contém 32 caracteres “0” ou “1”, representando uma instrução, seguidos por um comentário com a instrução assembly original para facilitar a depuração.

A implementação dessa etapa foi realizada no arquivo `assembler.c`, em uma estratégia de duas passagens. Essa abordagem é necessária para lidar com referências a rótulos (*labels*) de desvio que podem ser definidos depois de serem utilizados: uma primeira passagem resolve os endereços de todos os rótulos do programa, e uma segunda passagem efetivamente codifica cada instrução em binário, já podendo substituir cada referência simbólica pelo endereço correto.

4.4.1 Primeira Passagem: Resolução de Rótulos

A primeira passagem percorre o arquivo `codigo.asm` linha a linha, ignorando linhas vazias e comentários. Para cada linha lida, verifica-se o último caractere significativo: se for o caractere “:”, a linha é interpretada como a definição de um rótulo, e o par (nome, endereço atual de instrução) é registrado em uma tabela interna de rótulos; caso contrário, a linha é interpretada como uma instrução e armazenada em um vetor de instruções, com seu endereço incrementado em uma unidade. Ao final dessa passagem, dispõe-se de duas estruturas: a tabela de rótulos, que permitirá a tradução de referências simbólicas em endereços absolutos, e a sequência completa de instruções a serem codificadas.

4.4.2 Segunda Passagem: Codificação Binária

A segunda passagem percorre o vetor de instruções e, para cada uma, aplica a codificação binária correspondente ao formato de instrução adequado, conforme definido no conjunto de instruções do processador apresentado no Capítulo 2. O conjunto utiliza três formatos: Tipo R, para operações lógicas e aritméticas entre registradores; Tipo I, para operações imediatas, acessos à memória e desvios condicionais; e Tipo J, para desvios incondicionais e instruções de controle. A Tabela 6 resume a organização dos campos em cada formato.

A codificação propriamente dita é realizada por três funções auxiliares (`encode_r()`, `encode_i()` e `encode_j()`) que recebem os campos numéricos da instrução e os combinam por meio de operações de deslocamento e disjunção, produzindo a palavra de 32 bits final. As funções `split_line()` e `parse_mem()` encarregam-se de extrair, a partir do texto, o mnemônico, os operandos separados por vírgula e, no caso específico de LW e SW, o par *offset*/registrador-base no formato `offset($reg)`. A função `reg_num()` faz o mapeamento entre os nomes simbólicos dos registradores utilizados no assembly e seus respectivos índices físicos, conforme estabelecido na convenção da arquitetura: `$zero` é o registrador 0, `$v0` é o 2, os temporários `$t0`, `$t1` e `$t2` correspondem aos registradores 8, 9 e 10, o `$s0` é o 16, o ponteiro de pilha `$sp` é o 29 e o registrador de retorno `$ra` é o 31.

O tratamento de desvios merece atenção especial. Para instruções de desvio incondicional (J, JAL) o endereço alvo é absoluto: basta consultar a tabela de rótulos construída na primeira passagem e inserir o resultado diretamente no campo de 26 bits do formato Tipo J. Já para desvios condicionais (BEQ, BNE, BLT, BGT, BLE, BGE), o endereçamento é relativo ao próximo PC, conforme convenção MIPS. O *offset* a ser codificado no campo imediato de 16 bits é calculado como a diferença entre o endereço do rótulo alvo e o endereço da instrução seguinte:

$$\text{offset} = \text{endereço}(\text{label}) - (\text{endereço}(\text{instrução atual}) + 1)$$

Por exemplo, na sequência do código assembly do programa exemplo, a instrução

BLE \$t0, \$t1, L0 encontra-se no endereço 15, e o rótulo L0 é resolvido para o endereço 19; o *offset* codificado é, portanto, $19 - (15 + 1) = 3$, valor que aparece nos quatro bits menos significativos da instrução binária correspondente. Esse esquema é o que permite ao processador, ao executar a instrução, calcular o endereço de desvio simplesmente somando o *offset* ao PC já incrementado.

4.4.3 Saída Gerada

Como saída, a função `generate_binary()` escreve, no arquivo `codigo.mif`, uma linha para cada instrução, contendo a palavra binária de 32 bits seguida de um comentário com o endereço da instrução e seu texto original em assembly. Esse formato segue a convenção esperada por `$readmemb()`, que ignora os comentários e carrega apenas as palavras binárias na memória de instruções durante a simulação ou síntese em FPGA.

A título de exemplo, o código binário gerado para o programa apresentado nas seções anteriores é o seguinte:

```

1 00000100000111010000000011111111 // 0: ADDI $sp, $zero, 255
2 01000000000000000000000000000010 // 1: J main
3 00001011101111010000000000000001 // 2: SUBI $sp, $sp, 1
4 00011011101111110000000000000000 // 3: SW $ra, 0($sp)
5 00001011101111010000000000000111 // 4: SUBI $sp, $sp, 7
6 00110100000000100000000000000000 // 5: IN $v0
7 00011011101000100000000000000000 // 6: SW $v0, 0($sp)
8 00010111101010000000000000000000 // 7: LW $t0, 0($sp)
9 00011011101010000000000000000101 // 8: SW $t0, 5($sp)
10 00110100000000100000000000000000 // 9: IN $v0
11 00011011101000100000000000000001 // 10: SW $v0, 1($sp)
12 00010111101010000000000000000001 // 11: LW $t0, 1($sp)
13 00011011101010000000000000000110 // 12: SW $t0, 6($sp)
14 00010111101010000000000000000101 // 13: LW $t0, 5($sp)
15 00010111101010010000000000000110 // 14: LW $t1, 6($sp)
16 00101101000010010000000000000011 // 15: BLE $t0, $t1, L0
17 00010111101010000000000000000101 // 16: LW $t0, 5($sp)
18 00111001000000000000000000000000 // 17: OUT $t0
19 01000000000000000000000000000101 // 18: J L1
20 00010111101010000000000000000110 // 19: LW $t0, 6($sp)
21 00111001000000000000000000000000 // 20: OUT $t0
22 01001100000000000000000000000000 // 21: HALT

```

A correspondência entre o código assembly e o binário pode ser ilustrada inspecionando, por exemplo, a instrução `ADDI $sp, $zero, 255` na posição 0. Os 6 bits mais significativos (000001) correspondem ao *opcode* de `ADDI`; os 5 bits seguintes (00000) identificam o registrador `$zero` como *rs*; os 5 bits seguintes (11101) correspondem ao número 29, ou seja, ao registrador `$sp` como *rt*; e os 16 bits restantes (0000000011111111) codificam o valor imediato 255. Os demais formatos seguem o mesmo princípio, e o resultado final é um arquivo pronto para ser utilizado tanto em simulações em SystemVerilog quanto em implementações reais em FPGA, fechando o ciclo de compilação do código fonte em C-

até a máquina alvo.

4.5 Gerenciamento de Memória

A arquitetura alvo do compilador adota o modelo de Harvard, com memórias de instruções e de dados fisicamente separadas. O gerenciamento de memória durante a execução de um programa compilado, portanto, refere-se essencialmente à organização da memória de dados, que é dividida em duas grandes regiões: a região das variáveis globais, alocada de forma estática a partir do endereço zero, e a região da pilha (*stack*), utilizada para alocar dinamicamente as informações associadas a cada chamada de função.

As variáveis globais são alocadas pela função `scan_globals()` durante a primeira passagem do gerador de assembly. A cada variável global declarada na Tabela de Símbolos é atribuído um endereço sequencial a partir de zero, e a alocação de espaço considera o tamanho do identificador: variáveis escalares ocupam uma única posição de memória, enquanto vetores ocupam tantas posições quanto seu tamanho declarado. Os acessos a essas variáveis são realizados de forma direta no assembly, utilizando o registrador `$zero` como base, na forma `LW $reg, addr($zero)` ou `SW $reg, addr($zero)`.

A pilha, por outro lado, é gerenciada por meio do ponteiro `$sp` (*stack pointer*). No início da execução, `$sp` é inicializado com o valor 255, indicando o topo da memória de dados, e cresce em direção a endereços decrescentes a cada chamada de função, conforme a convenção de pilha descendente. A cada invocação, o prólogo da função aloca um *frame* (registro de ativação) na pilha, contendo, da base para o topo: o endereço de retorno (`$ra`), salvo na posição 0(`$sp`) para permitir o retorno após o término da função; as variáveis locais e os registradores temporários utilizados pela função, alocados em sequência conforme calculado pela função `scan_functions()`; e, acima desse bloco, os parâmetros formais, que foram empilhados pelo chamador antes da instrução `JAL`.

Os *offsets* de acesso a cada classe de operando são calculados por três funções auxiliares no `cgen.c`: `off_temp()` retorna o *offset* de um temporário a partir de seu índice; `off_local()` retorna o *offset* de uma variável local, considerando que as locais ficam logo após os temporários no *frame*; e `off_param()` retorna o *offset* de um parâmetro, considerando que os parâmetros ficam acima de todo o conteúdo interno do *frame*. Quando uma chamada de função empilha argumentos antes da instrução `JAL`, todos os *offsets* baseados em `$sp` dentro daquela sequência precisam ser ajustados pelo número de argumentos empilhados, ajuste recebido pelas funções `emit_load()` e `emit_store()` por meio do parâmetro `adj`.

Ao término da execução de uma função, o epílogo, gerado pela função `emit_epilogue()`, desfaz a alocação realizada pelo prólogo: restaura o registrador `$sp` somando-se a ele o tamanho do *frame* interno, restaura o endereço de retorno em `$ra` a partir da posição

`0($sp)`, libera o último *slot* da pilha e executa a instrução `JR $ra`, que devolve o controle ao chamador. No caso específico da função `main`, em vez do epílogo padrão, emite-se a instrução `HALT`, que finaliza a execução do programa. Essa organização garante o suporte adequado a chamadas aninhadas e recursivas, com cada ativação dispondo de seu próprio espaço de armazenamento isolado dos demais.

5 Exemplos

Neste capítulo são apresentados três programas escritos em C- que ilustram o funcionamento do compilador desenvolvido. Para cada um deles, mostra-se o código fonte, a Tabela de Símbolos construída pela fase de análise, o código intermediário gerado em quádruplas, o código assembly produzido pelo `cgen.c` e o código executável em formato binário gerado pelo `assembler.c`. Ao final de cada exemplo, discute-se a correspondência entre essas representações, evidenciando como cada construção da linguagem fonte é progressivamente traduzida para a arquitetura alvo.

Os três programas foram escolhidos para exercitar, em conjunto, todas as construções importantes da linguagem. O primeiro exemplo, o cálculo do fatorial, foca no uso de estruturas iterativas (`while`). O segundo, o algoritmo de Euclides para o cálculo do máximo divisor comum (MDC), explora chamadas recursivas e a manipulação da pilha de ativação. Por fim, o terceiro exemplo, um algoritmo de ordenação, exercita o uso de vetores e de chamadas de função com passagem por referência.

5.1 Exemplo 1 – Fatorial

O primeiro exemplo apresenta uma implementação recursiva do fatorial de um número inteiro. O programa, embora breve, exercita um conjunto representativo de construções da linguagem C-: declaração de função com tipo de retorno inteiro, parâmetro escalar passado por valor, comando `if` com retorno antecipado (*early return*), chamada recursiva, expressão aritmética envolvendo o resultado da chamada e, na função principal, leitura e escrita por meio das funções nativas `input()` e `output()`.

```
1  int fact (int n) {
2      if (n <= 0) {
3          return 1;
4      }
5      return n * fact(n - 1);
6  }
7
8  void main (void) {
9      int n;
10     n = input();
11     output(fact(n));
12 }
```

Listing 5.1 – Código fonte do programa Fatorial.

A função `fact` segue a definição matemática usual do fatorial, com o caso base sendo qualquer valor de `n` menor ou igual a zero, para o qual o resultado é a constante 1, e o caso recursivo sendo `n * fact(n - 1)`. Note-se que, embora a linguagem C- não imponha restrições sintáticas a comandos após um `return` dentro de um bloco `if`, optou-se aqui pelo padrão de retorno antecipado, em que o caso recursivo aparece fora do bloco condicional. Essa escolha permite observar como o compilador trata uma função com dois caminhos de retorno em pontos distintos do código.

Fase de Análise

A Tabela de Símbolos construída pelo compilador é apresentada a seguir.

===== TABELA DE SIMBOLOS =====				
LINHA	NOME	TIPO	CATEGORIA	ESCOPO
=====				
0	input	int	func	global
0	output	void	func	global
1	fact	int	func	global
8	main	void	func	global
1	n	int	var	fact
9	n	int	var	main
=====				

A tabela registra corretamente as duas funções declaradas pelo usuário e os parâmetros e variáveis associados aos seus respectivos escopos. Vale notar que existem duas declarações distintas da variável `n`, uma no escopo de `fact` (parâmetro formal) e outra no escopo de `main` (variável local). O fato de que a tabela registra ambas sem reportar conflito demonstra que o gerenciamento hierárquico de escopos descrito no Capítulo 3 está operando como esperado: a verificação de duplicidade é feita estritamente no escopo corrente, permitindo o sobreamento de nomes entre escopos distintos.

Código Intermediário

A partir da AST produzida pela fase de análise, o gerador de código intermediário produz a sequência de 23 quádruplas apresentada a seguir.

```

1  0: (FUNC, fact, -, -)
2  1: (PARAM_DECL, n, -, -)
3  2: (LDI, 0, -, $t0)
4  3: (<=, n, $t0, $t1)
5  4: (JZ, $t1, -, L0)

```



```

6  5: (LDI, 1, -, $t2)
7  6: (RET, $t2, -, -)
8  7: (LABEL, -, -, L0)
9  8: (LDI, 1, -, $t3)
10 9: (-, n, $t3, $t4)
11 10: (PARAM, $t4, -, -)
12 11: (CALL, fact, -, $t5)
13 12: (*, n, $t5, $t6)
14 13: (RET, $t6, -, -)
15 14: (END_FUNC, fact, -, -)
16 15: (FUNC, main, -, -)
17 16: (CALL, input, -, $t7)
18 17: (:=, $t7, -, n)
19 18: (PARAM, n, -, -)
20 19: (CALL, fact, -, $t8)
21 20: (PARAM, $t8, -, -)
22 21: (CALL, output, -, $t9)
23 22: (END_FUNC, main, -, -)

```

Listing 5.2 – Código intermediário gerado para o programa Fatorial.

A função `fact` é delimitada pelas quádruplas 0 e 14, com seu único parâmetro formal registrado pela quádrupla 1. O teste `n <= 0` aparece nas quádruplas 2 a 4: a constante zero é carregada em `$t0` por uma instrução `LDI`, a comparação relacional é realizada e seu resultado armazenado em `$t1`, e o desvio para o rótulo `L0` é registrado pela instrução `JZ` caso a condição seja falsa. O bloco `then`, contendo apenas o comando `return 1`, é traduzido pelas quádruplas 5 e 6, sendo a constante 1 carregada em `$t2` e em seguida retornada pela instrução `RET`.

Um ponto interessante a observar é que, diferentemente do que ocorre em estruturas `if-else` completas, não há aqui uma quádrupla `JMP` após o `RET`. Isso ocorre porque o gerador de código intermediário, ao perceber que o bloco `then` não é seguido por uma cláusula `else`, omite a instrução de salto que pularia sobre o bloco alternativo. O resultado é uma estrutura linear: caso a condição seja verdadeira, executa-se o `return 1`; caso seja falsa, o controle salta diretamente para o rótulo `L0`, a partir do qual o caso recursivo é executado normalmente.

O caso recursivo, iniciado no rótulo `L0` (quádrupla 7), começa pelo cálculo do argumento da chamada recursiva: a subtração `n - 1` é decomposta nas quádruplas 8 e 9, com a constante 1 carregada em `$t3` e o resultado da subtração armazenado em `$t4`. Esse temporário é empilhado como argumento pela quádrupla 10, e a chamada recursiva é realizada pela quádrupla 11, armazenando o retorno em `$t5`. A multiplicação `n * fact(n - 1)` é então realizada pela quádrupla 12, e o resultado é retornado pela quádrupla 13.

A função `main`, iniciada na quádrupla 15, traduz a leitura de entrada em uma sequência `CALL input` seguida de atribuição (quádruplas 16 e 17), e a expressão composta `output(fact(n))` aparece nas quádruplas 18 a 21, em que se observa o aninhamento de chamadas: primeiro empilha-se `n` como argumento de `fact`, faz-se a chamada armazenando o retorno em `$t8`, esse valor é então empilhado como argumento de `output` e, finalmente, é feita a chamada à função de saída.

Código Assembly

A tradução das quádruplas para o assembly da arquitetura alvo produz o código apresentado a seguir, com um total de 58 instruções.

```

1      ADDI $sp, $zero, 255
2      J main
3 fact:
4      SUBI $sp, $sp, 1
5      SW $ra, 0($sp)
6      SUBI $sp, $sp, 7
7      ADDI $t0, $zero, 0
8      SW $t0, 0($sp)
9      LW $t0, 8($sp)
10     LW $t1, 0($sp)
11     BGT $t0, $t1, L0
12     ADDI $t0, $zero, 1
13     SW $t0, 2($sp)
14     LW $v0, 2($sp)
15     ADDI $sp, $sp, 7
16     LW $ra, 0($sp)
17     ADDI $sp, $sp, 1
18     JR $ra
19 L0:
20     ADDI $t0, $zero, 1
21     SW $t0, 3($sp)
22     LW $t0, 8($sp)
23     LW $t1, 3($sp)
24     SUB $t2, $t0, $t1
25     SW $t2, 4($sp)
26     SUBI $sp, $sp, 1
27     LW $t0, 5($sp)
28     SW $t0, 0($sp)
29     JAL fact
30     ADDI $sp, $sp, 1

```

```
31      SW $v0, 5($sp)
32      LW $t0, 8($sp)
33      LW $t1, 5($sp)
34      MUL $t2, $t0, $t1
35      SW $t2, 6($sp)
36      LW $v0, 6($sp)
37      ADDI $sp, $sp, 7
38      LW $ra, 0($sp)
39      ADDI $sp, $sp, 1
40      JR $ra
41      ADDI $sp, $sp, 7
42      LW $ra, 0($sp)
43      ADDI $sp, $sp, 1
44      JR $ra
45 main:
46      SUBI $sp, $sp, 1
47      SW $ra, 0($sp)
48      SUBI $sp, $sp, 4
49      IN $v0
50      SW $v0, 0($sp)
51      LW $t0, 0($sp)
52      SW $t0, 3($sp)
53      SUBI $sp, $sp, 1
54      LW $t0, 4($sp)
55      SW $t0, 0($sp)
56      JAL fact
57      ADDI $sp, $sp, 1
58      SW $v0, 1($sp)
59      LW $t0, 1($sp)
60      OUT $t0
61      HALT
```

Listing 5.3 – Código assembly gerado para o programa Fatorial.

A correspondência entre o código intermediário e o assembly traz diversos pontos que merecem destaque. O primeiro é a fusão das quádruplas 3 e 4 ($\leq$ seguido de JZ) em uma única instrução BGT $\$t0, \$t1, L0$. Como o desvio para L0 ocorre exatamente quando a condição $n \leq 0$ é falsa, isto é, quando $n > 0$, o operador relacional é negado conforme o mecanismo descrito na Seção 4.3, eliminando a necessidade de calcular explicitamente o valor booleano da comparação.

O segundo ponto é o mecanismo de empilhamento diferido para a chamada recursiva, exatamente como descrito no exemplo do GCD. A quádrupla 10 (PARAM $\$t4$) não emite

código no momento em que é processada; quando a quádrupla 11 (`CALL fact`) é encontrada, o gerador pré-decrementa `$sp` em uma unidade (`SUBI $sp, $sp, 1`), empilha o argumento e emite a instrução `JAL fact`. Após o retorno, `$sp` é restaurado e o valor em `$v0` é armazenado na posição correspondente ao temporário `$t5`.

O terceiro ponto, especificamente interessante neste exemplo, é a presença de dois epílogos distintos dentro da função `fact`. O primeiro epílogo aparece logo após o bloco `then`, encerrando o caso base; o segundo aparece após o cálculo do produto, encerrando o caso recursivo. Adicionalmente, o gerador emite um terceiro epílogo após o rótulo `L1` implícito, garantindo que o controle nunca caia de uma função para outra mesmo nos casos em que todos os caminhos do programa já tenham retornado explicitamente. Embora esse epílogo final seja, na prática, código morto neste caso particular, sua presença é resultado da estratégia uniforme do gerador, que sempre emite o epílogo padrão ao final de cada função, antes do início da próxima.

Vale também observar que, dentro de `fact`, o parâmetro `n` é acessado por `LW $t0, 8($sp)`, isto é, com *offset* 8 a partir do topo da pilha. Esse valor é consistente com o tamanho do *frame* interno da função: a posição 0 é reservada ao endereço de retorno (`$ra`), as posições 0 a 6 (após o segundo `SUBI $sp, $sp, 7`) abrigam os sete temporários e variáveis locais usados, e a posição 8 corresponde ao primeiro *slot* acima desse bloco interno, onde o chamador deixou o argumento antes da instrução `JAL`.

Código Executável

Por fim, a montagem do assembly produz o arquivo `codigo.mif` com 58 palavras binárias de 32 bits, apresentado a seguir.

```

1 00000100000111010000000011111111 // 0: ADDI $sp, $zero, 255
2 0100000000000000000000000101010 // 1: J main
3 00001011101111010000000000000001 // 2: SUBI $sp, $sp, 1
4 00011011101111110000000000000000 // 3: SW $ra, 0($sp)
5 00001011101111010000000000000111 // 4: SUBI $sp, $sp, 7
6 00000100000010000000000000000000 // 5: ADDI $t0, $zero, 0
7 00011011101010000000000000000000 // 6: SW $t0, 0($sp)
8 00010111101010000000000000001000 // 7: LW $t0, 8($sp)
9 00010111101010010000000000000000 // 8: LW $t1, 0($sp)
10 00101001000010010000000000000111 // 9: BGT $t0, $t1, L0
11 00000100000010000000000000000001 // 10: ADDI $t0, $zero, 1
12 00011011101010000000000000000010 // 11: SW $t0, 2($sp)
13 00010111101000100000000000000010 // 12: LW $v0, 2($sp)
14 00000111101111010000000000000111 // 13: ADDI $sp, $sp, 7
15 00010111101111110000000000000000 // 14: LW $ra, 0($sp)
16 00000111101111010000000000000001 // 15: ADDI $sp, $sp, 1
17 00000011111000000000000000000111 // 16: JR $ra
18 00000100000010000000000000000001 // 17: ADDI $t0, $zero, 1
19 00011011101010000000000000000111 // 18: SW $t0, 3($sp)
20 00010111101010000000000000001000 // 19: LW $t0, 8($sp)
21 00010111101010010000000000000111 // 20: LW $t1, 3($sp)
22 00000001000010010101000000000001 // 21: SUB $t2, $t0, $t1

```

```

23 00011011101010100000000000000100 // 22: SW $t2, 4($sp)
24 00001011101111010000000000000001 // 23: SUBI $sp, $sp, 1
25 00010111101010000000000000000101 // 24: LW $t0, 5($sp)
26 00011011101010000000000000000000 // 25: SW $t0, 0($sp)
27 01000100000000000000000000000010 // 26: JAL fact
28 00000111101111010000000000000001 // 27: ADDI $sp, $sp, 1
29 000110111010001000000000000000101 // 28: SW $v0, 5($sp)
30 000101111010100000000000000001000 // 29: LW $t0, 8($sp)
31 000101111010100100000000000000101 // 30: LW $t1, 5($sp)
32 00000001000010010101000000000010 // 31: MUL $t2, $t0, $t1
33 00011011101010100000000000000110 // 32: SW $t2, 6($sp)
34 00010111101000100000000000000110 // 33: LW $v0, 6($sp)
35 00000111101111010000000000000111 // 34: ADDI $sp, $sp, 7
36 00010111101111110000000000000000 // 35: LW $ra, 0($sp)
37 00000111101111010000000000000001 // 36: ADDI $sp, $sp, 1
38 00000011111000000000000000000111 // 37: JR $ra
39 00000111101111010000000000000111 // 38: ADDI $sp, $sp, 7
40 00010111101111110000000000000000 // 39: LW $ra, 0($sp)
41 00000111101111010000000000000001 // 40: ADDI $sp, $sp, 1
42 00000011111000000000000000000111 // 41: JR $ra
43 00001011101111010000000000000001 // 42: SUBI $sp, $sp, 1
44 00011011101111110000000000000000 // 43: SW $ra, 0($sp)
45 00001011101111010000000000000100 // 44: SUBI $sp, $sp, 4
46 00110100000000100000000000000000 // 45: IN $v0
47 00011011101000100000000000000000 // 46: SW $v0, 0($sp)
48 00010111101010000000000000000000 // 47: LW $t0, 0($sp)
49 00011011101010000000000000000011 // 48: SW $t0, 3($sp)
50 00001011101111010000000000000001 // 49: SUBI $sp, $sp, 1
51 00010111101010000000000000000100 // 50: LW $t0, 4($sp)
52 00011011101010000000000000000000 // 51: SW $t0, 0($sp)
53 01000100000000000000000000000010 // 52: JAL fact
54 00000111101111010000000000000001 // 53: ADDI $sp, $sp, 1
55 00011011101000100000000000000001 // 54: SW $v0, 1($sp)
56 00010111101010000000000000000001 // 55: LW $t0, 1($sp)
57 00111001000000000000000000000000 // 56: OUT $t0
58 01001100000000000000000000000000 // 57: HALT

```

Listing 5.4 – Código executável (codigo.mif) gerado para o programa Fatorial.

A resolução de rótulos pelo `assembler.c` pode ser verificada inspecionando os campos de endereço alvo. Na instrução 1, `J main`, os 26 bits menos significativos codificam o valor 101010, ou seja, 42 em decimal, exatamente o endereço da primeira instrução da função `main` no arquivo montado. Nas instruções 26 e 52, ambas correspondentes a chamadas a `fact` (a primeira recursivamente na própria função e a segunda a partir de `main`), o campo de endereço alvo codifica o valor 2, que é o endereço da primeira instrução da função chamada. A presença de duas referências a `fact`, em pontos distintos do programa, ilustra a importância de o montador realizar a primeira passagem completa antes de iniciar a codificação: ambas as instruções precisam ser resolvidas para o mesmo endereço, ainda que sejam emitidas a partir de contextos diferentes.

A instrução 9, `BGT $t0, $t1, L0`, ilustra novamente o cálculo do *offset* de desvio condicional. O rótulo `L0` encontra-se no endereço 17, e o *offset* codificado nos 16 bits menos

significativos é $17 - (9 + 1) = 7$, valor que aparece nos três bits finais da instrução binária. Esse esquema permite que o processador calcule, em hardware, o endereço alvo do desvio simplesmente somando o *offset* ao PC já incrementado.

5.2 Exemplo 2 – GCD

O segundo exemplo implementa o algoritmo clássico de Euclides para o cálculo do máximo divisor comum entre dois inteiros, em uma versão recursiva. O programa, apresentado a seguir, é um exemplo padrão na literatura de compiladores (5) por exercitar simultaneamente funções com retorno de valor, chamadas recursivas, estruturas condicionais com ambos os ramos contendo **return** e expressões aritméticas encadeadas.

```
1  /* Um programa para calcular o mdc
2     segundo o algoritmo de Euclides. */
3
4  int gcd (int u, int v)
5  {
6      if (v == 0) return u;
7      else return gcd(v, u - u/v*v);
8      /* u-u/v*v == u mod v */
9  }
10
11 void main(void)
12 {
13     int x; int y;
14     x = input();
15     y = input();
16     output(gcd(x, y));
17 }
```

Listing 5.5 – Código fonte do programa GCD.

A estratégia do algoritmo é direta: se o segundo argumento é zero, o resultado é o primeiro; caso contrário, faz-se uma chamada recursiva trocando *u* por *v* e *v* por *u mod v*, expressão que aqui é calculada como *u - u/v*v*, uma vez que a linguagem C- não dispõe do operador de módulo. A função **main** apenas lê dois valores da entrada padrão, invoca **gcd** e imprime o resultado.

Fase de Análise

A compilação inicia-se pela fase de análise. Após a varredura léxica, sintática e semântica, e dada a ausência de erros, o compilador imprime a Tabela de Símbolos

apresentada a seguir.

===== TABELA DE SIMBOLOS =====				
LINHA	NOME	TIPO	CATEGORIA	ESCOPO
=====				
0	input	int	func	global
0	output	void	func	global
4	gcd	int	func	global
11	main	void	func	global
4	u	int	var	gcd
4	v	int	var	gcd
12	x	int	var	main
12	y	int	var	main
=====				

A tabela evidencia o correto registro das funções nativas `input()` e `output()`, inseridas no escopo global pela rotina de inicialização do compilador, bem como das funções `gcd` e `main` declaradas pelo programa. Os parâmetros `u` e `v` aparecem corretamente associados ao escopo da função `gcd`, e as variáveis locais `x` e `y` aparecem associadas ao escopo de `main`, demonstrando que o gerenciamento hierárquico de escopos descrito no Capítulo 3 está operando como esperado.

Código Intermediário

A partir da AST construída na fase de análise, o gerador de código intermediário produz a sequência de 29 quádruplas apresentada a seguir.

```

1  0: (FUNC, gcd, -, -)
2  1: (PARAM_DECL, u, -, -)
3  2: (PARAM_DECL, v, -, -)
4  3: (LDI, 0, -, $t0)
5  4: (==, v, $t0, $t1)
6  5: (JZ, $t1, -, L0)
7  6: (RET, u, -, -)
8  7: (JMP, -, -, L1)
9  8: (LABEL, -, -, L0)
10 9: (PARAM, v, -, -)
11 10: (/ , u, v, $t2)
12 11: (*, $t2, v, $t3)
13 12: (-, u, $t3, $t4)
14 13: (PARAM, $t4, -, -)
15 14: (CALL, gcd, -, $t5)

```

```

16 15: (RET, $t5, -, -)
17 16: (LABEL, -, -, L1)
18 17: (END_FUNC, gcd, -, -)
19 18: (FUNC, main, -, -)
20 19: (CALL, input, -, $t6)
21 20: (:=, $t6, -, x)
22 21: (CALL, input, -, $t7)
23 22: (:=, $t7, -, y)
24 23: (PARAM, x, -, -)
25 24: (PARAM, y, -, -)
26 25: (CALL, gcd, -, $t8)
27 26: (PARAM, $t8, -, -)
28 27: (CALL, output, -, $t9)
29 28: (END_FUNC, main, -, -)

```

Listing 5.6 – Código intermediário (quádruplas) gerado para o programa GCD.

A correspondência com o código fonte pode ser observada percorrendo as quádruplas em ordem. As quádruplas 0 a 2 delimitam o início da função `gcd` e registram seus dois parâmetros formais. A comparação `v == 0` do código fonte é traduzida pelas quádruplas 3 a 5: a constante zero é carregada em um temporário pela quádrupla LDI, a comparação propriamente dita é feita pela quádrupla `==` (que armazena seu resultado em `$t1`) e a quádrupla JZ desvia para o rótulo L0 caso o resultado seja falso, isto é, quando `v` é diferente de zero. O bloco `then`, contendo apenas o comando `return u`, é traduzido para a quádrupla 6, seguida por uma instrução JUMP que pula sobre o bloco `else`, conforme a estratégia descrita na Seção 4.2.

O bloco `else`, iniciado no rótulo L0, contém a chamada recursiva `gcd(v, u - u/v*v)`. A expressão aritmética `u - u/v*v` é decomposta em três operações elementares: a divisão `u/v` é avaliada primeiro (quádrupla 10, armazenando o resultado em `$t2`), em seguida o resultado é multiplicado por `v` (quádrupla 11, em `$t3`) e, por fim, esse produto é subtraído de `u` (quádrupla 12, em `$t4`). A ordem reflete a precedência usual dos operadores aritméticos: divisão e multiplicação são processadas antes da subtração, e a associatividade à esquerda garante que `u/v*v` seja avaliada como $(u/v)*v$. Os argumentos da chamada são empilhados nas quádruplas 9 e 13 e a chamada propriamente dita é registrada na quádrupla 14, com o resultado armazenado em `$t5` e retornado na quádrupla 15.

A função `main`, iniciada na quádrupla 18, traduz suas duas leituras de entrada em sequências `CALL input` seguidas de atribuições às variáveis `x` e `y` (quádruplas 19–22), e a expressão composta `output(gcd(x, y))` aparece nas quádruplas 23 a 27: primeiro empilham-se `x` e `y` como argumentos de `gcd`, então faz-se a chamada armazenando o retorno em `$t8`, esse valor é empilhado como argumento de `output` e, por fim, é feita a

chamada à função de saída.

Código Assembly

A tradução das quádruplas para o assembly da arquitetura alvo, realizada pelo `cgen.c`, produz o código apresentado a seguir.

```
1      ADDI $sp, $zero, 255
2      J   main
3
4 gcd:
5      SUBI $sp, $sp, 1
6      SW  $ra, 0($sp)
7      SUBI $sp, $sp, 6
8      ADDI $t0, $zero, 0
9      SW  $t0, 0($sp)
10     LW  $t0, 7($sp)
11     LW  $t1, 0($sp)
12     BNE $t0, $t1, L0
13     LW  $v0, 8($sp)
14     ADDI $sp, $sp, 6
15     LW  $ra, 0($sp)
16     ADDI $sp, $sp, 1
17     JR  $ra
18     J   L1
19 L0:
20     LW  $t0, 8($sp)
21     LW  $t1, 7($sp)
22     DIV $t2, $t0, $t1
23     SW  $t2, 2($sp)
24     LW  $t0, 2($sp)
25     LW  $t1, 7($sp)
26     MUL $t2, $t0, $t1
27     SW  $t2, 3($sp)
28     LW  $t0, 8($sp)
29     LW  $t1, 3($sp)
30     SUB $t2, $t0, $t1
31     SW  $t2, 4($sp)
32     SUBI $sp, $sp, 2
33     LW  $t0, 9($sp)
34     SW  $t0, 1($sp)
35     LW  $t0, 6($sp)
36     SW  $t0, 0($sp)
```

```
37     JAL gcd
38     ADDI $sp, $sp, 2
39     SW $v0, 5($sp)
40     LW $v0, 5($sp)
41     ADDI $sp, $sp, 6
42     LW $ra, 0($sp)
43     ADDI $sp, $sp, 1
44     JR $ra
45 L1:
46     ADDI $sp, $sp, 6
47     LW $ra, 0($sp)
48     ADDI $sp, $sp, 1
49     JR $ra
50
51 main:
52     SUBI $sp, $sp, 1
53     SW $ra, 0($sp)
54     SUBI $sp, $sp, 6
55     IN $v0
56     SW $v0, 0($sp)
57     LW $t0, 0($sp)
58     SW $t0, 4($sp)
59     IN $v0
60     SW $v0, 1($sp)
61     LW $t0, 1($sp)
62     SW $t0, 5($sp)
63     SUBI $sp, $sp, 2
64     LW $t0, 6($sp)
65     SW $t0, 1($sp)
66     LW $t0, 7($sp)
67     SW $t0, 0($sp)
68     JAL gcd
69     ADDI $sp, $sp, 2
70     SW $v0, 2($sp)
71     LW $t0, 2($sp)
72     OUT $t0
73     HALT
```

Listing 5.7 – Código assembly gerado para o programa GCD.

O assembly inicia-se com o prólogo do programa, comum a todos os exemplos: a instrução `ADDI $sp, $zero, 255` inicializa o ponteiro de pilha no topo da memória de

dados, e a instrução `J main` desvia o fluxo para o início da função principal. Cada função (`gcd` e `main`) começa com um prólogo que reserva espaço para o endereço de retorno e para o *frame* interno (temporários e variáveis locais), e termina com um epílogo que restaura `$ra` e devolve o controle ao chamador via `JR $ra`. No caso específico de `main`, o epílogo é substituído pela instrução `HALT`, que encerra a execução do processador.

A correspondência entre quádruplas e instruções assembly evidencia diversos pontos discutidos no Capítulo 4. O par de quádruplas 4 e 5 (`=` seguido de `JZ`) é traduzido para uma única instrução `BNE $t0, $t1, L0`: como o desvio para o caso falso ocorre exatamente quando os operandos da igualdade são diferentes, o operador relacional é negado e o resultado intermediário em `$t1` torna-se desnecessário. As três operações aritméticas que calculam $u - u/v*v$ (quádruplas 10 a 12) são traduzidas para blocos análogos de quatro instruções cada: dois `LW` para carregar os operandos, uma instrução aritmética (`DIV`, `MUL` e `SUB`) para computar o resultado e um `SW` para armazená-lo na posição do temporário de destino dentro do *frame* da função.

A chamada recursiva, talvez a parte mais sensível desse exemplo, ilustra o mecanismo de empilhamento diferido descrito na Seção 4.3. As quádruplas 9 e 13 (`PARAM v` e `PARAM $t4`) não emitem instruções no momento em que são processadas; apenas registram seus argumentos em um *buffer* interno. Quando a quádrupla 14 (`CALL gcd`) é encontrada, o gerador pré-decrementa o ponteiro de pilha em duas unidades (`SUBI $sp, $sp, 2`) e empilha os dois argumentos de uma só vez, na ordem correta, antes de emitir a instrução `JAL gcd`. Após o retorno, `$sp` é restaurado (`ADDI $sp, $sp, 2`) e o valor em `$v0`, contendo o retorno da chamada recursiva, é armazenado no *slot* correspondente ao temporário `$t5`. Esse mesmo padrão aparece em `main`, na chamada `gcd(x, y)`, demonstrando a uniformidade do tratamento de chamadas em todo o programa.

Vale também destacar como cada acesso a uma variável local ou parâmetro é traduzido em um par de instruções `LW/SW` com *offset* fixo a partir de `$sp`. Por exemplo, dentro de `gcd`, o parâmetro `v` é acessado por `LW $t0, 7($sp)` e o parâmetro `u` por `LW $t0, 8($sp)`: ambos estão acima do *frame* interno da função, que ocupa as posições 0 a 6, sendo `0($sp)` reservada ao endereço de retorno e as demais aos temporários e às locais conforme calculado pela função `scan_functions()`.

Código Executável

A tradução do assembly para binário, realizada pelo `assembler.c`, gera o arquivo `codigo.mif` apresentado a seguir, com 67 instruções de 32 bits cada. As 12 primeiras linhas são reproduzidas integralmente, e o restante do arquivo é apresentado em forma reduzida para economia de espaço; as instruções omitidas seguem o mesmo princípio de codificação descrito na Seção 4.4.

```
1 00000100000111010000000011111111 // 0: ADDI $sp, $zero, 255
```

[illegible]

```

59 00011011101010000000000000000001 // 58: SW $t0, 1($sp)
60 00010111101010000000000000000111 // 59: LW $t0, 7($sp)
61 00011011101010000000000000000000 // 60: SW $t0, 0($sp)
62 01000100000000000000000000000010 // 61: JAL gcd
63 00000111101111010000000000000010 // 62: ADDI $sp, $sp, 2
64 00011011101000100000000000000010 // 63: SW $v0, 2($sp)
65 00010111101010000000000000000010 // 64: LW $t0, 2($sp)
66 00111001000000000000000000000000 // 65: OUT $t0
67 01001100000000000000000000000000 // 66: HALT

```

Listing 5.8 – Código executável (`codigo.mif`) para o programa GCD.

A correspondência entre o assembly e o binário pode ser verificada inspecionando algumas instruções representativas. Na instrução 1, `J main`, os 6 bits mais significativos (010000) correspondem ao *opcode* de J, e os 26 bits restantes codificam o endereço alvo da instrução, no caso o valor 101101, que em decimal é 45, exatamente o endereço em que se inicia o rótulo `main` no programa. Da mesma forma, nas instruções 33 e 61, ambas correspondentes a chamadas a `gcd` (a primeira recursivamente na própria função e a segunda a partir de `main`), o campo de endereço alvo codifica o valor 2, que é o endereço da primeira instrução de `gcd`.

A instrução 9, `BNE $t0, $t1, L0`, ilustra o cálculo do *offset* de desvio condicional descrito na Seção 4.4. O rótulo `L0` encontra-se no endereço 16, e o *offset* codificado nos 16 bits menos significativos é $16 - (9 + 1) = 6$, valor que aparece nos quatro bits finais da instrução binária. Esse esquema permite que o processador calcule o endereço alvo de qualquer desvio condicional somando o *offset* ao PC já incrementado, mantendo a consistência da execução em hardware.

5.3 Exemplo 3 – Sort

O terceiro exemplo implementa um algoritmo clássico de ordenação por seleção (*Selection Sort*), aplicado a um vetor de dez elementos. Esse programa é particularmente interessante porque exercita construções que não apareceram nos exemplos anteriores: a declaração de variáveis globais (incluindo vetores), a passagem de vetores como parâmetros (passagem por referência), o uso de laços iterativos `while`, o uso de blocos com escopo próprio para variáveis locais e as operações de leitura e escrita em posições de vetor.

```

1 /* programa para ordenacao por selecao de
2    uma matriz com dez elementos. */
3 int vet[10];
4
5 int minloc (int a[], int low, int high) {
6     int i; int x; int k;
7     k = low;
8     x = a[low];

```

```
9      i = low + 1;
10     while (i < high) {
11         if (a[i] < x) {
12             x = a[i];
13             k = i;
14         }
15         i = i + 1;
16     }
17     return k;
18 }
19
20 void sort (int a[], int low, int high) {
21     int i; int k;
22     i = low;
23     while (i < high - 1) {
24         int t;
25         k = minloc(a, i, high);
26         t = a[k];
27         a[k] = a[i];
28         a[i] = t;
29         i = i + 1;
30     }
31 }
32
33 void main(void) {
34     int i;
35     i = 0;
36     while (i < 10) {
37         vet[i] = input();
38         i = i + 1;
39     }
40     sort(vet, 0, 10);
41     i = 0;
42     while (i < 10) {
43         output(vet[i]);
44         i = i + 1;
45     }
46 }
```

Listing 5.9 – Código fonte do programa Sort.

A estrutura do programa é a seguinte: a função `minloc` encontra a posição do

menor elemento em um subvetor delimitado pelos índices `low` e `high`; a função `sort` aplica iterativamente `minloc` e realiza a troca (*swap*) entre o menor elemento encontrado e o elemento na posição corrente; e a função `main` lê dez inteiros, invoca `sort` sobre o vetor global `vet`, e imprime o vetor ordenado.

Fase de Análise

A Tabela de Símbolos construída pelo compilador é apresentada a seguir.

===== TABELA DE SIMBOLOS =====				
LINHA	NOME	TIPO	CATEGORIA	ESCOPO
=====				
0	input	int	func	global
0	output	void	func	global
4	vet	int	vetor	global
6	minloc	int	func	global
21	sort	void	func	global
34	main	void	func	global
6	a	int	vetor	minloc
6	low	int	var	minloc
6	high	int	var	minloc
7	i	int	var	minloc
7	x	int	var	minloc
7	k	int	var	minloc
21	a	int	vetor	sort
21	low	int	var	sort
21	high	int	var	sort
22	i	int	var	sort
22	k	int	var	sort
25	t	int	var	sort:bloco
36	i	int	var	main
=====				

A tabela traz vários pontos dignos de nota. Em primeiro lugar, o vetor `vet` aparece corretamente registrado no escopo global, com categoria `vetor`, demonstrando que a regra gramatical de declaração de vetores e a correspondente verificação semântica estão funcionando. Em segundo, os parâmetros `a` de `minloc` e de `sort` são registrados também como `vetor`, refletindo o fato de que vetores são passados por referência na linguagem C-. Em terceiro, e mais interessante, a variável `t`, declarada dentro do bloco `while` de `sort`, aparece com escopo `sort:bloco`, evidenciando o correto tratamento de escopos aninhados

pela função `entra_escopo()` descrita no Capítulo 3: o compilador distingue o escopo da função do escopo do bloco interno, herdando o nome da função pai conforme a convenção definida na implementação.

Também vale observar que existem variáveis homônimas em escopos distintos sem conflito: `i` é declarada em `minloc`, em `sort` e em `main`; `a`, `low` e `high` aparecem como parâmetros tanto de `minloc` quanto de `sort`; e `k` aparece em `minloc` e em `sort`. O mecanismo de busca hierárquica de `busca_var()` garante que cada referência seja resolvida para a declaração visível mais próxima, conforme as regras de sombreamento (*shadowing*).

Código Intermediário

A partir da AST, o gerador de código intermediário produz a sequência de 88 quádruplas apresentada a seguir.

```

1  0: (FUNC, minloc, -, -)
2  1: (PARAM_DECL, a, -, -)
3  2: (PARAM_DECL, low, -, -)
4  3: (PARAM_DECL, high, -, -)
5  4: (:=, low, -, k)
6  5: (LDV, a[low], -, $t0)
7  6: (:=, $t0, -, x)
8  7: (LDI, 1, -, $t1)
9  8: (+, low, $t1, $t2)
10 9: (:=, $t2, -, i)
11 10: (LABEL, -, -, L0)
12 11: (<, i, high, $t3)
13 12: (JZ, $t3, -, L1)
14 13: (LDV, a[i], -, $t4)
15 14: (<, $t4, x, $t5)
16 15: (JZ, $t5, -, L2)
17 16: (LDV, a[i], -, $t6)
18 17: (:=, $t6, -, x)
19 18: (:=, i, -, k)
20 19: (LABEL, -, -, L2)
21 20: (LDI, 1, -, $t7)
22 21: (+, i, $t7, $t8)
23 22: (:=, $t8, -, i)
24 23: (JMP, -, -, L0)
25 24: (LABEL, -, -, L1)
26 25: (RET, k, -, -)
27 26: (END_FUNC, minloc, -, -)
28 27: (FUNC, sort, -, -)
29 28: (PARAM_DECL, a, -, -)
30 29: (PARAM_DECL, low, -, -)
31 30: (PARAM_DECL, high, -, -)
32 31: (:=, low, -, i)
33 32: (LABEL, -, -, L4)
34 33: (LDI, 1, -, $t9)
35 34: (-, high, $t9, $t10)
36 35: (<, i, $t10, $t11)
37 36: (JZ, $t11, -, L5)
38 37: (PARAM, a, -, -)

```



```

39 38: (PARAM, i, -, -)
40 39: (PARAM, high, -, -)
41 40: (CALL, minloc, -, $t12)
42 41: (:=, $t12, -, k)
43 42: (LDV, a[k], -, $t13)
44 43: (:=, $t13, -, t)
45 44: (LDV, a[i], -, $t14)
46 45: (:=, $t14, -, a[k])
47 46: (:=, t, -, a[i])
48 47: (LDI, 1, -, $t15)
49 48: (+, i, $t15, $t16)
50 49: (:=, $t16, -, i)
51 50: (JMP, -, -, L4)
52 51: (LABEL, -, -, L5)
53 52: (END_FUNC, sort, -, -)
54 53: (FUNC, main, -, -)
55 54: (LDI, 0, -, $t17)
56 55: (:=, $t17, -, i)
57 56: (LABEL, -, -, L6)
58 57: (LDI, 10, -, $t18)
59 58: (<, i, $t18, $t19)
60 59: (JZ, $t19, -, L7)
61 60: (CALL, input, -, $t20)
62 61: (:=, $t20, -, vet[i])
63 62: (LDI, 1, -, $t21)
64 63: (+, i, $t21, $t22)
65 64: (:=, $t22, -, i)
66 65: (JMP, -, -, L6)
67 66: (LABEL, -, -, L7)
68 67: (PARAM, vet, -, -)
69 68: (LDI, 0, -, $t23)
70 69: (PARAM, $t23, -, -)
71 70: (LDI, 10, -, $t24)
72 71: (PARAM, $t24, -, -)
73 72: (CALL, sort, -, $t25)
74 73: (LDI, 0, -, $t26)
75 74: (:=, $t26, -, i)
76 75: (LABEL, -, -, L8)
77 76: (LDI, 10, -, $t27)
78 77: (<, i, $t27, $t28)
79 78: (JZ, $t28, -, L9)
80 79: (LDV, vet[i], -, $t29)
81 80: (PARAM, $t29, -, -)
82 81: (CALL, output, -, $t30)
83 82: (LDI, 1, -, $t31)
84 83: (+, i, $t31, $t32)
85 84: (:=, $t32, -, i)
86 85: (JMP, -, -, L8)
87 86: (LABEL, -, -, L9)
88 87: (END_FUNC, main, -, -)

```

Listing 5.10 – Código intermediário gerado para o programa Sort.

A correspondência com o código fonte exige especial atenção a três construções inéditas. A primeira é a leitura de uma posição de vetor: a expressão `a[low]` no código

fonte (linha 7) é traduzida pela quádrupla 5, (LDV, a[low], -, \$t0), em que o operador LDV (*Load Vector*) abstrai o cálculo de endereço efetivo a ser realizado na fase de geração de assembly. A escrita em posição de vetor, por sua vez, aparece nas quádruplas 45, 46 e 61: a quádrupla 45, (:=, \$t14, -, a[k]), representa a atribuição $a[k] = a[i]$ do código de troca, e a quádrupla 61, (:=, \$t20, -, vet[i]), representa a leitura de um inteiro da entrada padrão diretamente para uma posição do vetor global.

A segunda construção destacável é a tradução do laço **while**, que segue o padrão descrito na Seção 4.2: um rótulo de início é fixado antes da avaliação da condição (quádruplas 10, 32, 56 e 75), a condição é avaliada e seu resultado testado por uma instrução JZ para um rótulo de saída (quádruplas 12, 36, 59 e 78), o corpo do laço é gerado, e um JMP incondicional retorna ao início. O exemplo permite observar quatro laços com essa estrutura, demonstrando a uniformidade do tratamento. Em **minloc**, o laço aninha ainda um comando **if** (quádruplas 13 a 19), cujo bloco **then** contém duas atribuições; como esse **if** não tem cláusula **else**, o gerador omite a quádrupla JMP para o final, conforme detalhado na implementação do `generate_quads()`.

A terceira construção é a passagem de vetor como argumento de função. Nas quádruplas 37 e 67, observa-se (PARAM, a, -, -) e (PARAM, vet, -, -), em que o argumento é o próprio identificador do vetor, e não uma expressão indexada. Essa distinção será fundamental na geração do assembly, em que o gerador precisará empilhar o endereço base do vetor em vez de um valor escalar, conforme controlado pela função `sym_is_array()` do `cgen.c`.

Código Assembly

A tradução para o assembly da arquitetura alvo produz 187 instruções, distribuídas entre os corpos das três funções **minloc**, **sort** e **main**. Em razão de sua extensão, a análise concentra-se na função **minloc**, que contém todas as construções relevantes do programa: acesso indexado a vetor, laço **while** e estrutura **if** sem **else**.

```

1      ADDI $sp, $zero, 255
2      J main
3 minloc:
4      SUBI $sp, $sp, 1
5      SW $ra, 0($sp)
6      SUBI $sp, $sp, 12
7      LW $t0, 14($sp)
8      SW $t0, 9($sp)
9      LW $s0, 14($sp)
10     LW $t0, 15($sp)
11     ADD $s0, $s0, $t0
12     LW $t0, 0($s0)
13     SW $t0, 0($sp)
14     LW $t0, 0($sp)
15     SW $t0, 10($sp)
16     ADDI $t0, $zero, 1

```

```

17     SW $t0, 1($sp)
18     LW $t0, 14($sp)
19     LW $t1, 1($sp)
20     ADD $t2, $t0, $t1
21     SW $t2, 2($sp)
22     LW $t0, 2($sp)
23     SW $t0, 11($sp)
24 L0:
25     LW $t0, 11($sp)
26     LW $t1, 13($sp)
27     BGE $t0, $t1, L1
28     LW $s0, 11($sp)
29     LW $t0, 15($sp)
30     ADD $s0, $s0, $t0
31     LW $t0, 0($s0)
32     SW $t0, 4($sp)
33     LW $t0, 4($sp)
34     LW $t1, 10($sp)
35     BGE $t0, $t1, L2
36     LW $s0, 11($sp)
37     LW $t0, 15($sp)
38     ADD $s0, $s0, $t0
39     LW $t0, 0($s0)
40     SW $t0, 6($sp)
41     LW $t0, 6($sp)
42     SW $t0, 10($sp)
43     LW $t0, 11($sp)
44     SW $t0, 9($sp)
45 L2:
46     ADDI $t0, $zero, 1
47     SW $t0, 7($sp)
48     LW $t0, 11($sp)
49     LW $t1, 7($sp)
50     ADD $t2, $t0, $t1
51     SW $t2, 8($sp)
52     LW $t0, 8($sp)
53     SW $t0, 11($sp)
54     J L0
55 L1:
56     LW $v0, 9($sp)
57     ADDI $sp, $sp, 12
58     LW $ra, 0($sp)
59     ADDI $sp, $sp, 1
60     JR $ra
61     ADDI $sp, $sp, 12
62     LW $ra, 0($sp)
63     ADDI $sp, $sp, 1
64     JR $ra
65 sort:
66     SUBI $sp, $sp, 1
67     SW $ra, 0($sp)
68     SUBI $sp, $sp, 11
69     LW $t0, 13($sp)
70     SW $t0, 8($sp)
71 L4:
72     ADDI $t0, $zero, 1
73     SW $t0, 0($sp)

```

```

74      LW $t0, 12($sp)
75      LW $t1, 0($sp)
76      SUB $t2, $t0, $t1
77      SW $t2, 1($sp)
78      LW $t0, 8($sp)
79      LW $t1, 1($sp)
80      BGE $t0, $t1, L5
81      SUBI $sp, $sp, 3
82      LW $t0, 17($sp)
83      SW $t0, 2($sp)
84      LW $t0, 11($sp)
85      SW $t0, 1($sp)
86      LW $t0, 15($sp)
87      SW $t0, 0($sp)
88      JAL minloc
89      ADDI $sp, $sp, 3
90      SW $v0, 3($sp)
91      LW $t0, 3($sp)
92      SW $t0, 9($sp)
93      LW $s0, 9($sp)
94      LW $t0, 14($sp)
95      ADD $s0, $s0, $t0
96      LW $t0, 0($s0)
97      SW $t0, 4($sp)
98      LW $t0, 4($sp)
99      SW $t0, 10($sp)
100     LW $s0, 8($sp)
101     LW $t0, 14($sp)
102     ADD $s0, $s0, $t0
103     LW $t0, 0($s0)
104     SW $t0, 5($sp)
105     LW $t0, 5($sp)
106     LW $s0, 9($sp)
107     LW $t1, 14($sp)
108     ADD $s0, $s0, $t1
109     SW $t0, 0($s0)
110     LW $t0, 10($sp)
111     LW $s0, 8($sp)
112     LW $t1, 14($sp)
113     ADD $s0, $s0, $t1
114     SW $t0, 0($s0)
115     ADDI $t0, $zero, 1
116     SW $t0, 6($sp)
117     LW $t0, 8($sp)
118     LW $t1, 6($sp)
119     ADD $t2, $t0, $t1
120     SW $t2, 7($sp)
121     LW $t0, 7($sp)
122     SW $t0, 8($sp)
123     J L4
124 L5:
125     ADDI $sp, $sp, 11
126     LW $ra, 0($sp)
127     ADDI $sp, $sp, 1
128     JR $ra
129 main:
130     SUBI $sp, $sp, 1

```

```

131     SW $ra, 0($sp)
132     SUBI $sp, $sp, 17
133     ADDI $t0, $zero, 0
134     SW $t0, 0($sp)
135     LW $t0, 0($sp)
136     SW $t0, 16($sp)
137 L6:
138     ADDI $t0, $zero, 10
139     SW $t0, 1($sp)
140     LW $t0, 16($sp)
141     LW $t1, 1($sp)
142     BGE $t0, $t1, L7
143     IN $v0
144     SW $v0, 3($sp)
145     LW $t0, 3($sp)
146     LW $s0, 16($sp)
147     ADDI $s0, $s0, 0
148     SW $t0, 0($s0)
149     ADDI $t0, $zero, 1
150     SW $t0, 4($sp)
151     LW $t0, 16($sp)
152     LW $t1, 4($sp)
153     ADD $t2, $t0, $t1
154     SW $t2, 5($sp)
155     LW $t0, 5($sp)
156     SW $t0, 16($sp)
157     J L6
158 L7:
159     ADDI $t0, $zero, 0
160     SW $t0, 6($sp)
161     ADDI $t0, $zero, 10
162     SW $t0, 7($sp)
163     SUBI $sp, $sp, 3
164     ADDI $t0, $zero, 0
165     SW $t0, 2($sp)
166     LW $t0, 9($sp)
167     SW $t0, 1($sp)
168     LW $t0, 10($sp)
169     SW $t0, 0($sp)
170     JAL sort
171     ADDI $sp, $sp, 3
172     SW $v0, 8($sp)
173     ADDI $t0, $zero, 0
174     SW $t0, 9($sp)
175     LW $t0, 9($sp)
176     SW $t0, 16($sp)
177 L8:
178     ADDI $t0, $zero, 10
179     SW $t0, 10($sp)
180     LW $t0, 16($sp)
181     LW $t1, 10($sp)
182     BGE $t0, $t1, L9
183     LW $s0, 16($sp)
184     ADDI $s0, $s0, 0
185     LW $t0, 0($s0)
186     SW $t0, 12($sp)
187     LW $t0, 12($sp)

```

```

188     OUT $t0
189     ADDI $t0, $zero, 1
190     SW $t0, 14($sp)
191     LW $t0, 16($sp)
192     LW $t1, 14($sp)
193     ADD $t2, $t0, $t1
194     SW $t2, 15($sp)
195     LW $t0, 15($sp)
196     SW $t0, 16($sp)
197     J L8
198 L9:
199     HALT

```

Listing 5.11 – Código assembly gerado para o programa Sort.

A correspondência mais importante a destacar é a tradução das quádruplas LDV em sequências de instruções que computam o endereço efetivo do elemento acessado. Considere-se a quádrupla 5, (LDV, `a[low]`, `-`, `$t0`), traduzida nas instruções correspondentes ao acesso à posição `a[low]`:

```

1     LW $s0, 14($sp)    # carrega o endereço base do vetor "a"
2     LW $t0, 15($sp)    # carrega o valor do índice "low"
3     ADD $s0, $s0, $t0  # calcula o endereço efetivo a + low
4     LW $t0, 0($s0)     # le o conteúdo da posicao a[low]

```

Essa sequência ilustra o mecanismo de passagem por referência adotado pelo compilador: o parâmetro formal `a` de `minloc` contém, na pilha, o endereço base do vetor (e não uma cópia de seus elementos), e o acesso indexado é realizado somando-se a esse endereço o valor do índice. A função auxiliar `emit_load()` do `cgen.c` encapsula essa lógica e a aplica uniformemente sempre que um identificador de vetor é encontrado no padrão `nome[idx]`.

A escrita em posição de vetor segue o mesmo princípio, mas usando uma instrução `SW` ao invés de `LW` ao final da sequência. Para acessos ao vetor global `vet` dentro de `main` (não mostrados neste trecho), o endereço base é o próprio endereço estático atribuído na primeira passagem do `scan_globals()`, e a soma é feita com base no registrador `$zero`, em vez de carregar o endereço base da pilha. Esse comportamento aparece, por exemplo, na escrita `vet[i] = input()` dentro do primeiro laço de `main` e nas leituras durante a impressão final.

Vale observar também a correspondência das instruções relacionais: a quádrupla 11, (`<`, `i`, `high`, `$t3`), seguida da quádrupla 12, (`JZ`, `$t3`, `-`, `L1`), é traduzida para a única instrução `BGE $t0, $t1, L1`, em que o operador relacional é negado conforme o mecanismo de fusão descrito na Seção 4.3: o desvio para o final do laço deve ocorrer quando $i \geq \text{high}$, ou seja, quando a condição original $i < \text{high}$ se torna falsa.

Código Executável

O `assembler.c` traduz o assembly em um arquivo `codigo.mif` com 187 palavras binárias de 32 bits. Reproduzem-se a seguir as primeiras instruções do arquivo, contendo o prólogo do programa e o início da função `minloc`, junto com as instruções de chamada e fim que ilustram a resolução de rótulos no programa completo.

```

1 00000100000111010000000011111111 // 0: ADDI $sp, $zero, 255
2 01000000000000000000000000000001 // 1: J main
3 00001011101111010000000000000001 // 2: SUBI $sp, $sp, 1
4 00011011101111111000000000000000 // 3: SW $ra, 0($sp)
5 000010111011110100000000000001100 // 4: SUBI $sp, $sp, 12
6 000101111010100000000000000001110 // 5: LW $t0, 14($sp)
7 000110111010100000000000000001001 // 6: SW $t0, 9($sp)
8 000101111011000000000000000001110 // 7: LW $s0, 14($sp)
9 000101111010100000000000000001111 // 8: LW $t0, 15($sp)
10 00000010000010001000000000000000 // 9: ADD $s0, $s0, $t0
11 00010110000010000000000000000000 // 10: LW $t0, 0($s0)
12 00011011101010000000000000000000 // 11: SW $t0, 0($sp)
13 00010111101010000000000000000000 // 12: LW $t0, 0($sp)
14 000110111010100000000000000001010 // 13: SW $t0, 10($sp)
15 00000100000010000000000000000001 // 14: ADDI $t0, $zero, 1
16 00011011101010000000000000000001 // 15: SW $t0, 1($sp)
17 000101111010100000000000000001110 // 16: LW $t0, 14($sp)
18 00010111101010010000000000000001 // 17: LW $t1, 1($sp)
19 00000001000010010101000000000000 // 18: ADD $t2, $t0, $t1
20 00011011101010100000000000000010 // 19: SW $t2, 2($sp)
21 00010111101010000000000000000010 // 20: LW $t0, 2($sp)
22 000110111010100000000000000001011 // 21: SW $t0, 11($sp)
23 000101111010100000000000000001011 // 22: LW $t0, 11($sp)
24 000101111010100100000000000001101 // 23: LW $t1, 13($sp)
25 001100010000100100000000000011010 // 24: BGE $t0, $t1, L1
26 000101111011000000000000000001011 // 25: LW $s0, 11($sp)
27 000101111010100000000000000001111 // 26: LW $t0, 15($sp)
28 00000010000010001000000000000000 // 27: ADD $s0, $s0, $t0
29 00010110000010000000000000000000 // 28: LW $t0, 0($s0)
30 00011011101010000000000000000100 // 29: SW $t0, 4($sp)
31 00010111101010000000000000000100 // 30: LW $t0, 4($sp)
32 000101111010100100000000000001010 // 31: LW $t1, 10($sp)
33 001100010000100100000000000001001 // 32: BGE $t0, $t1, L2
34 000101111011000000000000000001011 // 33: LW $s0, 11($sp)
35 000101111010100000000000000001111 // 34: LW $t0, 15($sp)
36 00000010000010001000000000000000 // 35: ADD $s0, $s0, $t0
37 00010110000010000000000000000000 // 36: LW $t0, 0($s0)
38 00011011101010000000000000000110 // 37: SW $t0, 6($sp)
39 00010111101010000000000000000110 // 38: LW $t0, 6($sp)
40 000110111010100000000000000001010 // 39: SW $t0, 10($sp)
41 000101111010100000000000000001011 // 40: LW $t0, 11($sp)
42 000110111010100000000000000001001 // 41: SW $t0, 9($sp)
43 00000100000010000000000000000001 // 42: ADDI $t0, $zero, 1
44 00011011101010000000000000000111 // 43: SW $t0, 7($sp)
45 000101111010100000000000000001011 // 44: LW $t0, 11($sp)
46 00010111101010010000000000000111 // 45: LW $t1, 7($sp)
47 00000001000010010101000000000000 // 46: ADD $t2, $t0, $t1
48 000110111010101000000000000001000 // 47: SW $t2, 8($sp)
49 000101111010100000000000000001000 // 48: LW $t0, 8($sp)

```

```

50 00011011101010000000000000001011 // 49: SW $t0, 11($sp)
51 010000000000000000000000000010110 // 50: J L0
52 00010111101000100000000000001001 // 51: LW $v0, 9($sp)
53 00000111101111010000000000001100 // 52: ADDI $sp, $sp, 12
54 00010111101111110000000000000000 // 53: LW $ra, 0($sp)
55 00000111101111010000000000000001 // 54: ADDI $sp, $sp, 1
56 00000011111000000000000000000111 // 55: JR $ra
57 00000111101111010000000000001100 // 56: ADDI $sp, $sp, 12
58 00010111101111110000000000000000 // 57: LW $ra, 0($sp)
59 00000111101111010000000000000001 // 58: ADDI $sp, $sp, 1
60 00000011111000000000000000000111 // 59: JR $ra
61 00001011101111010000000000000001 // 60: SUBI $sp, $sp, 1
62 00011011101111110000000000000000 // 61: SW $ra, 0($sp)
63 00001011101111010000000000000101 // 62: SUBI $sp, $sp, 11
64 00010111101010000000000000001101 // 63: LW $t0, 13($sp)
65 00011011101010000000000000001000 // 64: SW $t0, 8($sp)
66 00000100000010000000000000000001 // 65: ADDI $t0, $zero, 1
67 00011011101010000000000000000000 // 66: SW $t0, 0($sp)
68 00010111101010000000000000001100 // 67: LW $t0, 12($sp)
69 00010111101010010000000000000000 // 68: LW $t1, 0($sp)
70 00000001000010010101000000000001 // 69: SUB $t2, $t0, $t1
71 00011011101010100000000000000001 // 70: SW $t2, 1($sp)
72 00010111101010000000000000001000 // 71: LW $t0, 8($sp)
73 00010111101010010000000000000001 // 72: LW $t1, 1($sp)
74 00110001000010010000000000101011 // 73: BGE $t0, $t1, L5
75 00001011101111010000000000000111 // 74: SUBI $sp, $sp, 3
76 00010111101010000000000000010001 // 75: LW $t0, 17($sp)
77 0001101110101000000000000000010 // 76: SW $t0, 2($sp)
78 00010111101010000000000000001011 // 77: LW $t0, 11($sp)
79 00011011101010000000000000000001 // 78: SW $t0, 1($sp)
80 00010111101010000000000000001111 // 79: LW $t0, 15($sp)
81 00011011101010000000000000000000 // 80: SW $t0, 0($sp)
82 0100010000000000000000000000010 // 81: JAL minloc
83 00000111101111010000000000000111 // 82: ADDI $sp, $sp, 3
84 00011011101000100000000000000111 // 83: SW $v0, 3($sp)
85 00010111101010000000000000000111 // 84: LW $t0, 3($sp)
86 00011011101010000000000000001001 // 85: SW $t0, 9($sp)
87 00010111101100000000000000001001 // 86: LW $s0, 9($sp)
88 00010111101010000000000000001110 // 87: LW $t0, 14($sp)
89 00000010000010001000000000000000 // 88: ADD $s0, $s0, $t0
90 00010110000010000000000000000000 // 89: LW $t0, 0($s0)
91 0001101110101000000000000000100 // 90: SW $t0, 4($sp)
92 0001011110101000000000000000100 // 91: LW $t0, 4($sp)
93 00011011101010000000000000001010 // 92: SW $t0, 10($sp)
94 00010111101100000000000000001000 // 93: LW $s0, 8($sp)
95 00010111101010000000000000001110 // 94: LW $t0, 14($sp)
96 00000010000010001000000000000000 // 95: ADD $s0, $s0, $t0
97 00010110000010000000000000000000 // 96: LW $t0, 0($s0)
98 00011011101010000000000000000101 // 97: SW $t0, 5($sp)
99 00010111101010000000000000000101 // 98: LW $t0, 5($sp)
100 00010111101100000000000000001001 // 99: LW $s0, 9($sp)
101 00010111101010010000000000001110 // 100: LW $t1, 14($sp)
102 00000010000010011000000000000000 // 101: ADD $s0, $s0, $t1
103 00011010000010000000000000000000 // 102: SW $t0, 0($s0)
104 00010111101010000000000000001010 // 103: LW $t0, 10($sp)
105 00010111101100000000000000001000 // 104: LW $s0, 8($sp)
106 00010111101010010000000000001110 // 105: LW $t1, 14($sp)

```



```

107 0000001000000100110000000000000000 // 106: ADD $s0, $s0, $t1
108 0001101000000100000000000000000000 // 107: SW $t0, 0($s0)
109 0000010000000100000000000000000000 // 108: ADDI $t0, $zero, 1
110 00011011101010000000000000000000110 // 109: SW $t0, 6($sp)
111 000101111010100000000000000000001000 // 110: LW $t0, 8($sp)
112 00010111101010010000000000000000110 // 111: LW $t1, 6($sp)
113 000000010000010010101000000000000000 // 112: ADD $t2, $t0, $t1
114 00011011101010100000000000000000111 // 113: SW $t2, 7($sp)
115 00010111101010000000000000000000111 // 114: LW $t0, 7($sp)
116 000110111010100000000000000000001000 // 115: SW $t0, 8($sp)
117 0100000000000000000000000000000001000001 // 116: J L4
118 000001111011110100000000000000001011 // 117: ADDI $sp, $sp, 11
119 000101111011111100000000000000000000 // 118: LW $ra, 0($sp)
120 000001111011110100000000000000000001 // 119: ADDI $sp, $sp, 1
121 0000000111110000000000000000000000111 // 120: JR $ra
122 000010111011110100000000000000000001 // 121: SUBI $sp, $sp, 1
123 000110111011111100000000000000000000 // 122: SW $ra, 0($sp)
124 000010111011110100000000000000000001 // 123: SUBI $sp, $sp, 17
125 000001000000010000000000000000000000 // 124: ADDI $t0, $zero, 0
126 000110111010100000000000000000000000 // 125: SW $t0, 0($sp)
127 000101111010100000000000000000000000 // 126: LW $t0, 0($sp)
128 000110111010100000000000000000000000 // 127: SW $t0, 16($sp)
129 00000100000001000000000000000000001010 // 128: ADDI $t0, $zero, 10
130 000110111010100000000000000000000001 // 129: SW $t0, 1($sp)
131 000101111010100000000000000000000000 // 130: LW $t0, 16($sp)
132 000101111010100100000000000000000001 // 131: LW $t1, 1($sp)
133 0011000100000100100000000000000001111 // 132: BGE $t0, $t1, L7
134 001101000000000100000000000000000000 // 133: IN $v0
135 000110111010000100000000000000000001 // 134: SW $v0, 3($sp)
136 000101111010100000000000000000000001 // 135: LW $t0, 3($sp)
137 000101111011000000000000000000000000 // 136: LW $s0, 16($sp)
138 000001100000100000000000000000000000 // 137: ADDI $s0, $s0, 0
139 000110100000010000000000000000000000 // 138: SW $t0, 0($s0)
140 000001000000010000000000000000000001 // 139: ADDI $t0, $zero, 1
141 000110111010100000000000000000000000 // 140: SW $t0, 4($sp)
142 000101111010100000000000000000000000 // 141: LW $t0, 16($sp)
143 000101111010100100000000000000000000 // 142: LW $t1, 4($sp)
144 000000010000010010101000000000000000 // 143: ADD $t2, $t0, $t1
145 0001101110101010000000000000000000101 // 144: SW $t2, 5($sp)
146 0001011110101000000000000000000000101 // 145: LW $t0, 5($sp)
147 000110111010100000000000000000000000 // 146: SW $t0, 16($sp)
148 010000000000000000000000000000000000 // 147: J L6
149 000001000000010000000000000000000000 // 148: ADDI $t0, $zero, 0
150 0001101110101000000000000000000000110 // 149: SW $t0, 6($sp)
151 00000100000001000000000000000000001010 // 150: ADDI $t0, $zero, 10
152 0001101110101000000000000000000000111 // 151: SW $t0, 7($sp)
153 000010111011110100000000000000000001 // 152: SUBI $sp, $sp, 3
154 000001000000010000000000000000000000 // 153: ADDI $t0, $zero, 0
155 0001101110101000000000000000000000010 // 154: SW $t0, 2($sp)
156 000101111010100000000000000000000000 // 155: LW $t0, 9($sp)
157 000110111010100000000000000000000001 // 156: SW $t0, 1($sp)
158 00010111101010000000000000000000001010 // 157: LW $t0, 10($sp)
159 000110111010100000000000000000000000 // 158: SW $t0, 0($sp)
160 01000100000000000000000000000000000111100 // 159: JAL sort
161 000001111011110100000000000000000001 // 160: ADDI $sp, $sp, 3
162 000110111010000100000000000000000000 // 161: SW $v0, 8($sp)
163 000001000000010000000000000000000000 // 162: ADDI $t0, $zero, 0

```

```

164 0001101110101000000000000000001001 // 163: SW $t0, 9($sp)
165 0001011110101000000000000000001001 // 164: LW $t0, 9($sp)
166 0001101110101000000000000000001000 // 165: SW $t0, 16($sp)
167 0000010000001000000000000000001010 // 166: ADDI $t0, $zero, 10
168 0001101110101000000000000000001010 // 167: SW $t0, 10($sp)
169 0001011110101000000000000000001000 // 168: LW $t0, 16($sp)
170 0001011110101001000000000000001010 // 169: LW $t1, 10($sp)
171 0011000100001001000000000000001111 // 170: BGE $t0, $t1, L9
172 0001011110110000000000000000001000 // 171: LW $s0, 16($sp)
173 0000011000010000000000000000000000 // 172: ADDI $s0, $s0, 0
174 0001011000001000000000000000000000 // 173: LW $t0, 0($s0)
175 0001101110101000000000000000001100 // 174: SW $t0, 12($sp)
176 0001011110101000000000000000001100 // 175: LW $t0, 12($sp)
177 0011100100000000000000000000000000 // 176: OUT $t0
178 0000010000001000000000000000000001 // 177: ADDI $t0, $zero, 1
179 0001101110101000000000000000001110 // 178: SW $t0, 14($sp)
180 0001011110101000000000000000001000 // 179: LW $t0, 16($sp)
181 0001011110101001000000000000001110 // 180: LW $t1, 14($sp)
182 0000000100001001010100000000000000 // 181: ADD $t2, $t0, $t1
183 0001101110101010000000000000001111 // 182: SW $t2, 15($sp)
184 0001011110101000000000000000001111 // 183: LW $t0, 15($sp)
185 0001101110101000000000000000001000 // 184: SW $t0, 16($sp)
186 0100000000000000000000000010100110 // 185: J L8
187 0100110000000000000000000000000000 // 186: HALT

```

Listing 5.12 – Código executável (`codigo.mif`) gerado para o programa Sort.

A resolução dos rótulos pelo montador pode ser verificada inspecionando os campos de endereço alvo das instruções tipo J. Na instrução 1, J `main`, os 26 bits menos significativos codificam o valor 1111001, ou seja, 121 em decimal, exatamente o endereço da primeira instrução da função `main`. Na instrução 81, JAL `minloc`, o campo de endereço alvo carrega o valor 2, correspondente ao início de `minloc`. E na instrução 159, JAL `sort`, o campo carrega o valor 111100, ou seja, 60 em decimal, exatamente onde o rótulo `sort` é definido no programa montado. Esses três valores demonstram que o esquema de resolução de rótulos em duas passagens do `assembler.c` funciona corretamente mesmo quando uma função (no caso, `minloc`) é referenciada antes da definição de outra (`sort`), uma vez que a primeira passagem coleta todos os rótulos antes de qualquer codificação ser realizada.

Os três exemplos apresentados neste capítulo, em conjunto, demonstram que o compilador desenvolvido é capaz de tratar adequadamente o subconjunto da linguagem C- proposto: estruturas iterativas, recursão, vetores, expressões aritméticas e relacionais, chamadas com passagem por valor e por referência, e blocos com escopo próprio. O fluxo completo de compilação, da análise léxica até a geração do arquivo `.mif` pronto para a FPGA, foi exercitado em programas de complexidade crescente, e os resultados intermediários produzidos em cada etapa confirmam a correteza da implementação.

6 Conclusão

O projeto desenvolvido ao longo da disciplina alcançou seu objetivo central de implementar um compilador funcional para a linguagem C-, integrando-se ao processador MIPS monociclo desenvolvido na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores. O fluxo completo de compilação foi realizado em todos os exemplos apresentados no Capítulo 5, partindo do código fonte de alto nível e percorrendo as fases de análise léxica, com o auxílio do Flex, análise sintática e semântica, com o auxílio do Bison, geração de código intermediário em quádruplas, geração de código assembly para a arquitetura do processador e, por fim, montagem do código binário no formato `.mif`, para ser carregado na memória de instruções do processador. O resultado é a integração completa de iniciar em um programa escrito em uma linguagem próxima do programador transformando-se, por meio de etapas sucessivas de tradução, em um executável para a arquitetura projetada no laboratório anterior.

6.1 Dificuldades Encontradas

A principal dificuldade enfrentada durante o desenvolvimento foi o tratamento da passagem de vetores por referência, necessário para viabilizar a implementação do algoritmo de ordenação apresentado no Exemplo 3. Diferentemente da passagem de parâmetros escalares, que se resolve por uma simples cópia de valor para a pilha, a passagem de vetores exigiu gerenciar dinamicamente o endereço base recebido como parâmetro e somá-lo ao índice da iteração para compor o endereço efetivo de cada acesso. A solução adotada consistiu em utilizar o registrador `$s0` como auxiliar dedicado ao cálculo de endereços, preservando os registradores de rascunho `$t0`, `$t1` e `$t2` para a manipulação dos valores propriamente ditos. Essa separação de papéis exigiu cuidado adicional na geração do código de instruções como LDV e atribuições indexadas, e foi um dos pontos do projeto que demandaram maior atenção tanto na fase de modelagem quanto na de implementação.

6.2 Destaques

Um dos destaques do projeto é a arquitetura de *memory spilling* adotada no gerador de código assembly: em vez de tentar alocar dezenas de temporários produzidos pelo gerador de código intermediário em um conjunto fisicamente limitado de registradores do processador, optou-se por utilizar a memória RAM como apoio para todos os temporários, empregando os registradores `$t0`, `$t1` e `$t2` estritamente como rascunhos no padrão *load* $\rightarrow$ *compute* $\rightarrow$ *store*. Essa decisão de projeto protegeu o compilador contra o esgotamento

de registradores, permitindo que programas complexos pudessem ser compilados sem a necessidade de mecanismos de alocação. Outro destaque é a otimização de fusão de desvios implementada no gerador de assembly: ao detectar o padrão recorrente de uma quádrupla relacional seguida imediatamente por uma instrução JZ, o gerador funde as duas operações em uma única instrução de desvio condicional do processador (BGE, BLE, BEQ, entre outras), negando a lógica do operador relacional original. Essa fusão economiza ciclos de *clock* e ocupação de memória de instrução, e exemplifica como otimizações simples, mas bem direcionadas, podem trazer ganhos significativos sem comprometer a clareza do código gerado.

6.3 Trabalhos Futuros

Uma possível futura adição no projeto é a implementação de um algoritmo clássico de alocação de registradores, como o de coloração de grafos, em substituição à estratégia atual de *spilling* integral para a memória. Tal mecanismo permitiria reduzir o volume de instruções LW e SW emitidas pelo gerador, melhorando a performance de execução do programa no processador. Outra ideia é a introdução de fases dedicadas de otimização independentes de máquina, atuando sobre a Árvore Sintática Abstrata ou sobre o vetor de quádruplas, com técnicas como propagação de constantes, eliminação de subexpressões comuns e eliminação de código morto.

Referências

- 1 ALFRED, V. et al. *Compilers Principles, Techniques*. [S.l.]: Pearson, 2007. Citado 3 vezes nas páginas 7, 29 e 50.
- 2 PATTERSON, D.; HENNESSY, J. L. *Organização e projeto de computadores*. 5. ed. Rio de Janeiro: Elsevier, 2017. Citado na página 9.
- 3 FPGA4STUDENT. *Verilog code for clock divider on FPGA*. 2017. Acesso em: jul. 2025. Disponível em: <<https://www.fpga4student.com/2017/08/verilog-code-for-clock-divider-on-fpga.html>>. Citado na página 15.
- 4 HARTMAN, K. *Encoding MIPS Instructions with C++17*. 2020. Disponível em: <https://medium.com/@kevin_hartman/encoding-mips-instructions-with-c-17-59f9c2170faa>. Citado na página 24.
- 5 LOUDEN, K. C. *Compiladores-Princípios e Práticas*. [S.l.]: Cengage Learning Editores, 2004. Citado 2 vezes nas páginas 34 e 68.